Ingeniería y Calidad de Software

Resumen para el Segundo Parcial

UNIDAD 2 – Gestión Lean-Agile de Productos de Software

Filosofía Lean

Lean es una filosofía **más vieja que Agile**, nacida en Japón en los años '40 con **Toyota**. Busca **maximizar el valor para el cliente con el mínimo consumo de recursos**: todo esfuerzo que no crea valor es **desperdicio**. Es el **cliente** quien determina si lo que se entrega tiene valor o no. Persigue alta calidad, bajo costo y poco tiempo de entrega, produciendo lo que el cliente quiere, cuando lo quiere y donde lo quiere.

Usa el método **Just in Time (JIT)**: entregar o intercambiar el producto entre fases justo antes de que se use y en la cantidad necesaria. En software: *"analice cuando lo necesite, no antes"*.

Los principios originales eran para manufactura; el matrimonio **Poppendieck** (*Lean Software Development*) los adaptó al software porque este es un producto intangible.

Los 7 principios Lean

1. Eliminar desperdicios

Es **el más importante**. Todo paso que no aporta valor es desperdicio (aunque el proceso nunca es perfecto: siempre se puede mejorar). Se quiere evitar producir funcionalidades que luego no se usan (Pareto: el 80% del valor está en el 20% de las funcionalidades), el retrabajo y que los artefactos queden obsoletos antes de terminarse. Se relaciona con los principios ágiles 3, 7 y 10 (entregar software funcionando, que sea la medida de progreso y maximizar el trabajo no realizado). En software, el trabajo a medias y la funcionalidad extra son el "inventario" de la manufactura.

Los 7 desperdicios del software (entre corchetes, su equivalente en manufactura):

- **Características extras** [productos extra]: funcionalidades de más y también la *Up Front Specification* (especificar por adelantado), porque cuando se implementa, esa información ya está obsoleta, incompleta o errónea. Se relaciona con el principio 4 (diferir compromisos).
- **Trabajo a medias** [acumulación de stock]: US, casos de uso o bugs sin terminar que hay que retomar. Por eso Agile usa **gestión binaria** (está terminado o no): estimar un % de avance es un desperdicio en sí mismo.
- **Proceso extra**: pasos del proceso que no aportan valor al producto. Los procesos definidos sobrecargaron a las personas con tareas de poco valor y, como resistencia, surgieron los procesos empíricos.
- **Búsqueda de información**: sin la disciplina de SCM no hay información accesible ni trazabilidad de los ítems de configuración, lo que dificulta ver el impacto de los cambios y ralentiza la resolución de errores.
- **Defectos**: errores que se arrastran a una etapa posterior a la que se introdujeron y generan retrabajo (Testing "devuelve" la funcionalidad a desarrollo). El antídoto es el **Aseguramiento de Calidad**.
- **Esperas** [costos de logística]: tiempo que una persona espera para poder trabajar (depende de otro, necesita una aprobación, etc.). Por eso se busca formar **equipos multidisciplinarios**.
- **Cambios de tareas (multitasking)**: al ser un trabajo intelectual, cambiar de tarea hace perder tiempo de preparación (*tiempo de seteo*). Por eso Kanban **limita el WIP**. Esperas y cambios de tareas están ligados al JIT.

2. Amplificar el aprendizaje

Transparencia y transformar el conocimiento **implícito en explícito**. Crear una cultura de mejora continua donde las personas intercambian experiencias y todo lo aprendido se comparte con toda la organización. Se relaciona con aceptar cambios de requerimientos en etapas finales y con que personas técnicas y no técnicas trabajen juntas todo el proyecto.

3. Embeber la integridad conceptual

La calidad **no se negocia** y la simplicidad es clave. El producto debe satisfacer las expectativas y necesidades del cliente, y todas las partes deben encajar con coherencia y consistencia, de modo que los requerimientos se vean como un todo armónico.

4. Diferir compromisos hasta el último momento responsable

Postergar las decisiones lo suficiente como para tener la mayor información posible, pero antes de que sea demasiado tarde. Se difieren las **decisiones irreversibles**. En Agile se refleja en el **Product Backlog**, que nunca tiene el 100% de los requerimientos y se completa a medida que hay más información. Las mejores estrategias de diseño dejan opciones abiertas.

5. Dar poder al equipo

Libertad de acción y poder de decisión para un equipo **multifuncional y autogestionado**, capacitado y motivado (principio 11 del Manifiesto Ágil). Implica no subordinar al equipo desde arriba, entrenar líderes y delegar decisiones, y a la vez que los miembros se comprometan a cumplir lo pactado. Es el **talón de Aquiles** de Lean y Agile: en la práctica pocas veces se reúnen todas esas características.

6. Ver el todo

Visión holística: entender el producto, el valor que hay detrás y el servicio completo, de modo que los objetivos particulares de cada área estén alineados con los globales.

7. Entregar lo antes posible

Entregar software funcionando y útil rápido, antes de que cambien las necesidades del negocio. Se acotan los ciclos de desarrollo y se entregan **incrementos pequeños de valor**, saliendo pronto al mercado con un producto mínimo valioso.

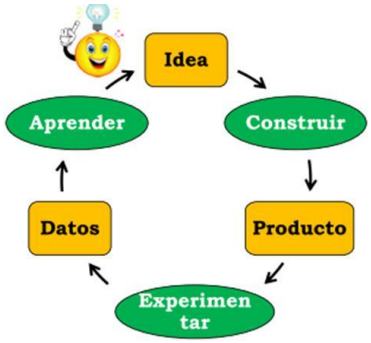
Relación Lean – Ágil

Lean es **anterior** al Manifiesto Ágil y varios principios ágiles heredan de él. Ambos están orientados al cliente y al máximo valor, entienden que los equipos deben estar motivados (la sinergia es valor agregado), valoran la flexibilidad ante el cambio y buscan productos de calidad con mejora continua.

Lean UX

> **En palabras simples:** Lean UX es aplicar el pensamiento Lean al diseño de la experiencia de usuario. En vez de pasar meses diseñando algo "perfecto" y descubrir al final que a nadie le sirve, se construye algo mínimo, se prueba con usuarios reales y se aprende rápido. Equivocarse al principio sale barato; equivocarse al final sale carísimo.

Es un proceso **iterativo** basado en el ciclo **Build – Measure – Learn** (construir, medir, aprender; también llamado *Build–Experiment–Learn*): se saca una conclusión a partir de usuarios reales que usan (o van a usar) el producto. Si desde el primer momento el usuario no "aprueba" el producto, es más barato **pivotar** (cambiar de rumbo) y reiniciar el ciclo. **El éxito no es entregar un producto, sino entregar un producto o característica que el cliente efectivamente use**; por eso se alinean continuamente los esfuerzos con las necesidades reales del cliente.



Ciclo Idea – Construir – Producto – Experimentar – Datos – Aprender

Scrum (base necesaria para escalar)

Scrum es un marco **ligero** que ayuda a personas, equipos y organizaciones a generar valor mediante soluciones adaptables a **problemas complejos**. En pocas palabras: el **Product Owner** ordena el trabajo en un **Product Backlog**; el equipo convierte una selección de ese trabajo en un **Incremento** de valor durante un **Sprint**; el equipo y los stakeholders inspeccionan el resultado y ajustan para el próximo Sprint; se repite. El marco es *deliberadamente incompleto*: solo define lo necesario y se apoya en la inteligencia colectiva. Se basa en el **empirismo** (el conocimiento viene de la experiencia y de decidir según lo observado) y en el **pensamiento Lean** (reducir desperdicios, foco en lo esencial). Es iterativo e incremental para optimizar la previsibilidad y controlar el riesgo.

Pilares empíricos:

- **Transparencia**: el proceso y el trabajo deben ser visibles para quienes lo hacen y lo reciben. Sin ella, la inspección genera engaños y desperdicio.
- **Inspección**: artefactos y progreso se revisan con frecuencia y diligencia para detectar desvíos. Sin adaptación, la inspección es inútil.
- **Adaptación**: si algo se desvía de límites aceptables, se ajusta lo antes posible. Es más difícil si las personas no están empoderadas o no pueden autogestionarse.

Valores: Compromiso, Enfoque, Apertura, Respeto y Coraje.

Equipo Scrum: Scrum Master, Product Owner y Desarrolladores, sin sub-equipos ni jerarquías. Es **multifuncional** (tiene todas las habilidades) y **autogestionado** (decide internamente quién hace qué, cuándo y cómo). Normalmente **10 personas o menos**. Si crece demasiado se divide en varios equipos que comparten el mismo objetivo de producto, Product Backlog y Product Owner.

- **Desarrolladores**: crean el Sprint Backlog, instalan calidad cumpliendo la Definición de Hecho, adaptan su plan cada día hacia el Sprint Goal y se responsabilizan mutuamente.
- **Product Owner**: maximiza el valor del producto y gestiona el Product Backlog (comunica el Objetivo del Producto, crea y ordena los ítems y asegura que sea transparente). Es **una persona, no un comité**; quien quiera cambiar el backlog debe negociar con él.
- **Scrum Master**: responsable de que Scrum se entienda y se aplique y de la efectividad del equipo. Es un líder servidor: capacita en autogestión, elimina impedimentos, asegura que los eventos se hagan y respeten su time-box, ayuda al PO con técnicas de backlog y asesora a la organización en la adopción de Scrum.

Eventos (todos con **time-box**; el Sprint contiene a los demás):

- **Sprint**: duración fija de **un mes o menos**; uno comienza apenas termina el anterior. Durante el Sprint no se hacen cambios que pongan en riesgo el Sprint Goal, la calidad no baja, el backlog se refina y el alcance se puede renegociar con el PO. Solo el **PO** puede cancelarlo (si el Sprint Goal queda obsoleto). Si no se llega al final, **se entrega menos en vez de extender el tiempo** (se negocia alcance, no tiempo ni costo).
- **Sprint Planning** (máx. 8 hs para un Sprint de un mes): responde tres temas: *¿por qué es valioso este Sprint?* (se define el Sprint Goal), *¿qué se puede hacer?* (los desarrolladores seleccionan ítems del backlog según su capacidad y rendimiento pasado) y *¿cómo se hará?* (descomponen en tareas de un día o menos). Sprint Goal + ítems elegidos + plan = **Sprint Backlog**.
- **Daily Scrum** (15 min máx.): para los desarrolladores, mismo lugar y hora todos los días. Inspecciona el progreso hacia el Sprint Goal y adapta el Sprint Backlog. Mejora la comunicación, expone impedimentos y evita otras reuniones.
- **Sprint Review** (máx. 4 hs): inspecciona el resultado del Sprint con los stakeholders y decide adaptaciones. Es una sesión de trabajo, no una simple presentación.
- **Sprint Retrospective** (máx. 3 hs): cierra el Sprint. Planifica mejoras de calidad y eficacia analizando personas, interacciones, procesos, herramientas y Definición de Hecho.
- **Refinamiento del Product Backlog**: actividad continua (sin tiempo fijo, se expresa como % del Sprint) de descomponer y detallar ítems (descripción, orden, tamaño). Los desarrolladores son responsables del tamaño.

Artefactos y su compromiso:

- **Product Backlog** → compromiso: **Objetivo del Producto** (estado futuro del producto, objetivo a largo plazo). Lista ordenada y emergente; única fuente de trabajo del equipo.
- **Sprint Backlog** → compromiso: **Sprint Goal** (el único objetivo del Sprint, da flexibilidad sobre el trabajo exacto). Compuesto por el por qué (goal), el qué (ítems) y el cómo (plan). Es un plan por y para los desarrolladores y se actualiza durante el Sprint.
- **Incremento** → compromiso: **Definición de Hecho (DoD)**. Paso concreto hacia el Objetivo del Producto, aditivo y verificado; debe ser utilizable. Un ítem que no cumple la DoD no se libera ni se muestra en la Review: vuelve al Product Backlog. Si varios equipos trabajan en un producto, comparten la misma DoD.

DoR y DoD: la **Definición de Listo (DoR)** indica cuándo una US está lo bastante bien formulada para entrar a un Sprint (ej.: tiene prototipo). La **DoD** es el checklist que indica cuándo está lo bastante bien implementada para mostrarse en la Review y, si el PO la aprueba, pasar a producción.

Capacidad: métrica de Scrum que determina cuánto trabajo puede comprometer el equipo en un Sprint; se define en el Planning y se contrasta con las US del Product Backlog. En equipos maduros se estima en Story Points; en menos maduros, en horas ideales.

Niveles de planificación: *Portfolio* (1 año o más; stakeholders y PO), *Producto* (varios meses; visión y roadmap), *Release* (3 a 9 meses; equipo entero; balancea valor, calidad y restricciones de alcance, cronograma y presupuesto), *Iteración* (1 semana a 1 mes; qué entregar y cómo) y *Día* (diaria; equipo de desarrollo).

Herramientas: la **tarjeta de tarea** (código del ítem de backlog, nombre de la actividad, iniciales del responsable, día del sprint y estimación de tiempo) y el **tablero** con columnas *Story, To Do, Work in Process, To Verify y Done* (conviene granularidad fina de tareas). Los **gráficos de backlog** muestran el progreso: *Burn-Down* (curva descendente: trabajo que queda), *Burn-Up* (curva ascendente: trabajo completado a lo largo de la release) y *Combined Burn Chart* (ambos).

Ejemplo para entender DoR vs. DoD: en un equipo que desarrolla una app de turnos médicos, la US "Reservar turno" cumple la **DoR** si tiene el prototipo de pantalla aprobado y criterios de aceptación claros (puede entrar al Sprint). Cumple la **DoD** si el código está revisado, pasó las pruebas unitarias y de integración, tiene cero defectos conocidos y está en el repositorio (puede mostrarse en la Sprint Review).

PREGUNTA DE PARCIAL ▸ Dados los principios del Manifiesto Ágil, explique cómo el Framework Scrum 2020 propone aplicar esos principios. Justifique.

Cómo pensarla: tomá cada principio ágil y buscá qué evento, rol o artefacto de Scrum lo hace posible. Si la preguntan, recorré los 12 en orden.

1. **Satisfacción del cliente con entrega temprana y continua**: Sprints de duración fija y creación de Incrementos cuando se cumple la Definición de Terminado.
2. **Aceptar cambios en los requisitos**: se fomentan a través de las Sprint Review, con colaboración y retroalimentación del cliente (y el Product Backlog puede reordenarse).
3. **Entrega frecuente de software funcional**: Sprints de **hasta un mes**, o sea entregas en tiempos cortos y frecuentes.
4. **Colaboración entre negocio y desarrolladores**: los stakeholders participan con el equipo en las Sprint Review, apoyados por el Scrum Master.
5. **Individuos motivados**: los equipos son **autogestionados**, deciden roles, tiempos y métodos de trabajo.
6. **Comunicación cara a cara**: Daily Scrums y Sprint Review, y el Scrum Master elimina barreras de comunicación.
7. **El software funcionando como medida de progreso**: cada Sprint entrega Incrementos utilizables y verificados que aportan al Objetivo del Producto.
8. **Desarrollo sostenible**: ciclos de Sprint consistentes generan un ritmo de trabajo constante.
9. **Excelencia técnica**: las Retrospectivas y los equipos multifuncionales mejoran calidad y agilidad.
10. **Simplicidad (minimizar el trabajo innecesario)**: Definición de Terminado, refinamiento del Product Backlog y eventos optimizados (time-box).
11. **Diseños emergentes de equipos autoorganizados**: los equipos autogestionados determinan sus flujos de trabajo de forma autónoma.
12. **Reflexión y ajuste regulares**: las Retrospectivas del Sprint aseguran mejora continua en calidad y efectividad.

Escalado de Scrum: Framework Nexus

Scrum limita el tamaño del equipo (≈10 personas), por lo que para productos más grandes surge la necesidad de **escalarlo**. Existen varios frameworks (Nexus, Scale Scrum, LeSS, SAFe); en esta materia se estudia **Nexus**.

Nexus es un framework de roles, eventos, artefactos y técnicas que vincula el trabajo de **3 a 9 equipos Scrum** que trabajan sobre **un único Product Backlog común** con **un solo Product Owner**, para construir un **incremento integrado "Terminado"**. Surge para manejar la complejidad de las **dependencias entre equipos**: de **requerimientos** (el alcance puede superponerse y la forma de implementar afecta a otros) y de **conocimiento del dominio** (hay que mapearlo a los equipos para minimizar interrupciones durante el Sprint).

Roles / responsabilidades:

- **Nexus Integration Team (NIT)**: asegura que se produzca un **incremento integrado al menos una vez por Sprint**, abordando restricciones técnicas y no técnicas que lo impidan. Lo integran el PO, un Scrum Master y uno o más miembros de los equipos que ayudan a adoptar herramientas y prácticas.
- **Product Owner**: maximiza el valor del trabajo integrado de todos los equipos y gestiona el único Product Backlog.

- **Scrum Master**: asegura que Nexus se entienda y aplique según su guía (puede ser SM de uno o más equipos del Nexus).

Eventos (Nexus agrega o extiende los de Scrum; su duración se guía por la de los eventos Scrum correspondientes):

- **Nexus Sprint**: los equipos producen un solo Integrated Increment (igual que Scrum).
- **Refinamiento entre equipos** (continuo): descompone el Product Backlog para hacer **transparentes y eliminar o minimizar las dependencias**, y ayuda a prever qué equipo entregará qué ítems.
- **Nexus Sprint Planning**: coordina las actividades de todos los equipos para un Sprint; participan representantes de cada equipo y el PO. Resultados: Sprint Goal de cada equipo, **Objetivo del Nexus**, un **Nexus Sprint Backlog** y el Sprint Backlog de cada equipo.
- **Nexus Daily Scrum**: asisten solo **representantes** de cada equipo; se discuten problemas de integración y se inspecciona el progreso hacia el objetivo del Nexus. La Daily de cada equipo la complementa.
- **Nexus Sprint Review**: se hace al final del Sprint para dar feedback sobre el Integrated Increment y **reemplaza** a las Sprint Reviews individuales.
- **Nexus Sprint Retrospective**: planifica mejoras para todo el Nexus; complementa a las retrospectivas de cada equipo ("inteligencia de abajo hacia arriba").

Artefactos:

- **Product Backlog** único, entendido a un nivel que permita detectar y minimizar dependencias; compromiso: Objetivo del Producto.
- **Nexus Sprint Backlog**: Objetivo del Sprint del Nexus + ítems de cada equipo; resalta dependencias y flujo de trabajo. Compromiso: **Objetivo de Sprint del Nexus** (suma de los objetivos de los equipos).
- **Integrated Increment**: suma de todo el trabajo integrado completado por el Nexus. Compromiso: **Definición de Terminado**. Está terminado solo si está **integrado, es de valor y utilizable**. El NIT define una DoD aplicable a cada Sprint; cada equipo puede ser más estricto, **nunca menos riguroso**.

> **En palabras simples:** si un solo equipo Scrum (≈10 personas) no alcanza para el producto, no se hace un "equipo gigante": se juntan **3 a 9 equipos Scrum** que comparten **un solo Product Backlog y un solo Product Owner**, y se agrega un equipo extra, el **Nexus Integration Team**, cuya única misión es que lo que hacen todos los equipos **se integre** en un incremento que funcione al final de cada Sprint.

Ejemplo: una empresa desarrolla una app de delivery y el cliente quiere salir a producción pronto. Arma 3 equipos Scrum: uno para la app de clientes, otro para la app de repartidores y otro para el panel del comercio. Hay **un único Product Backlog** ordenado por un solo PO. Los tres equipos dependen entre sí (por ejemplo, el pedido del cliente lo ve el repartidor), así que esas **dependencias** se detectan en el refinamiento entre equipos y se resuelven en la Nexus Daily Scrum.

> **PREGUNTA DE PARCIAL ▸ Presente una propuesta de cómo estructuraría el equipo de proyecto para que 3 equipos trabajen en paralelo utilizando un framework para escalar Scrum. (la respuesta es básicamente la guía de Nexus)**

Cómo responderla (estructura sugerida a partir de la guía de Nexus del apunte):

- **Framework elegido:** Nexus, porque está pensado para 3 a 9 equipos Scrum sobre **un único Product Backlog** y un solo PO, y se enfoca en resolver las dependencias entre equipos.
- **Estructura:** 3 Scrum Teams (cada uno con su Scrum Master o compartido, y sus desarrolladores) + 1 **Product Owner** único + **Nexus Integration Team** (PO, un Scrum Master y miembros de los equipos que ayudan a adoptar herramientas y prácticas).
- **Eventos:** Refinamiento entre equipos (continuo) → Nexus Sprint Planning (resultado: objetivo de cada equipo, Objetivo del Nexus, Nexus Sprint Backlog) → Nexus Daily Scrum (solo representantes, foco en integración) → Nexus Sprint Review (reemplaza las reviews de cada equipo) → Nexus Sprint Retrospective (complementa las de cada equipo).
- **Artefactos:** Product Backlog único (compromiso: Objetivo del Producto), Nexus Sprint Backlog (compromiso: Objetivo de Sprint del Nexus) e **Integrated Increment** (compromiso: Definición de Terminado; los equipos pueden ser más estrictos pero nunca menos).
- **Justificación:** el cliente quiere un producto en producción cercana: los 3 equipos trabajan en paralelo y el NIT garantiza **al menos un incremento integrado por Sprint**, sin multiplicar los backlogs ni los POs.

Método Kanban en el contexto del desarrollo de software

Kanban no es un proceso de desarrollo de software ni una metodología de administración de proyectos. Es un **enfoque para gestionar las formas de trabajo (procesos) con el objetivo de lograr mejora continua**. Su principio es **"empieza por donde estés"**: no plantea cambios revolucionarios sino mejoras graduales sobre el proceso o metodología que la organización ya tiene, lo que **reduce la resistencia al cambio**. Surge de estudios de **Toyota** sobre almacenamiento y tiempo de stockeo en supermercados y, al igual que Lean, fue necesario adaptarlo al software.

Aprovecha los principios Lean: define el valor desde la perspectiva del cliente, **limita el trabajo en proceso (WIP)**, identifica y elimina desperdicios, identifica y elimina las **barreras en el flujo** (todo lo que atrasa el proceso; se relaciona con entregar lo antes posible) y promueve una cultura de mejora continua.

Prácticas de Kanban

- **Visualizar el trabajo**: el trabajo requerido por el cliente se divide en **piezas de trabajo** (US, features, bugs, temas, épicas, cambios) escritas en **tarjetas señalizadoras (kan-bans)** ubicadas en un **tablero**. Las tarjetas indican cuándo una pieza se puede "pullear" y en qué parte del proceso está. El tablero es un sistema de flujo de izquierda a derecha; cada etapa es una **columna** (sobre las que se aplica la teoría de colas). Da **transparencia** porque está siempre a la vista. Hay dos tipos de columnas: de **producción** (piezas en las que se está trabajando) y de **acumulación** (piezas listas para pasar a la siguiente etapa, base del sistema de arrastre).
- **Limitar el WIP**: se fija un **límite concreto de piezas en progreso por columna** para evitar atascos y cuellos de botella. Genera un sistema **pull**: el trabajo se "arrastra" cuando otro se completa y queda capacidad, en vez de "empujarlo" al paso siguiente. Mucho trabajo sin terminar es desperdicio de tiempo y dinero y alarga los tiempos de entrega; limitarlo mejora calidad, tiempo y tasa de entrega.
- **Gestionar el flujo de trabajo**: el trabajo fluye por un **tablero permanente**; **no existe el concepto de iteración, proceso o proyecto**. El tablero puede compartirse con otros equipos y proyectos. Se busca un flujo continuo e ininterrumpido y se analiza para detectar cuellos de botella, recursos ociosos, tiempos de entrega y de espera.
- **Hacer explícitas las políticas**: deben ser **escasas, simples, bien definidas, visibles, siempre aplicadas y fácilmente modificables** (si resultan contraproducentes se cambian). Se visualizan, por ejemplo, con resúmenes entre columnas de lo que debe estar hecho para mover una tarjeta, y con el WIP de cada columna. Las políticas de calidad o DoD deben estar definidas, publicadas y promovidas.
- **Mejorar y evolucionar**: no introduce cambios grandes; identifica el proceso, lo hace explícito para todos y propone mejoras que se aplican gradualmente.
- **Circuito de retroalimentación**: parte esencial de cualquier proceso controlado. Hay que definir la frecuencia de las reuniones: demasiado frecuentes obligan a cambiar cosas que no se vieron con los cambios anteriores; poco frecuentes dan bajo rendimiento durante mucho tiempo.

¿Cómo aplicar Kanban?

Se empieza con lo que se tiene (entender el proceso actual); se identifican las **unidades de trabajo** (US, CU, defectos); se identifican las **clases de servicio** (distintos trabajos con distintas políticas/DoD: requerimientos, defectos, desarrollo, solicitudes); se **visualiza el flujo** diseñando el tablero con columnas; se **definen políticas por clase de servicio** (WIP por columna, colores de tarjetas, capacidad destinada, fechas de entrega); y se **identifican y resuelven cuellos de botella** (asignando más recursos o ajustando el WIP).

Métricas, valores y principios

Las métricas representativas son **Cycle Time, Lead Time, Touch Time y Eficiencia del Ciclo de Proceso** (explicadas en la sección de métricas).

Los **valores** se resumen en una palabra: **respeto**, que es la base de los otros ocho: **transparencia** (compartir información abiertamente mejora el flujo de valor), **equilibrio** (entre demanda y capacidad, por ejemplo), **colaboración**, **foco en el cliente**, **flujo**, **liderazgo** (necesario en todos los niveles), **entendimiento** (conocerse a sí mismo para mejorar, el punto de partida), **acuerdo** (compromiso dinámico de avanzar juntos; no es gestión por consenso) y **respeto**.

Principios directores: *sostenibilidad* (foco sostenible y en la mejora), *orientación al servicio* (rendimiento y satisfacción del cliente) y *supervivencia* (competitividad y adaptabilidad).

Principios fundacionales (seis, en dos grupos). *Gestión del cambio*: empezar con lo que estás haciendo ahora (entender los procesos actuales tal como se hacen y respetar roles, responsabilidades y puestos); acordar buscar la mejora a través del **cambio evolutivo**; fomentar el **liderazgo en cada nivel**. *Despliegue de servicio*: entender y focalizarse en las

necesidades y expectativas de los clientes; gestionar el trabajo y dejar que la gente se **autoorganice** en torno a las tareas; **evolucionar las políticas** para mejorar los resultados.

Scrum vs. Kanban

Scrum trabaja con **iteraciones de tiempo fijo** y el equipo asume un **compromiso** por iteración; Kanban no necesita tiempo fijo (la cadencia puede variar, marcada por eventos) y el compromiso es opcional. En Scrum la métrica por defecto es la **velocidad**; en Kanban es el **Lead Time**. Scrum exige equipos **multifuncionales**, Kanban admite también especializados. Scrum limita el WIP **indirectamente** (lo marca el Sprint); Kanban lo limita **directamente** en cada etapa. En Scrum no se agrega alcance una vez iniciado el Sprint, en Kanban se puede agregar trabajo siempre que haya capacidad. Scrum requiere **estimaciones**, Kanban las deja opcionales. En Scrum cada equipo tiene su backlog y el tablero **se limpia en cada Sprint**; en Kanban todos los equipos pueden compartir la pizarra y el tablero es **persistente**. Scrum **prescribe tres roles** (PO, SM, Equipo), Kanban ninguno; en Scrum el backlog debe estar priorizado y las funcionalidades deben caber en un Sprint, en Kanban la priorización es opcional y no se prescribe el tamaño. Kanban no tiene concepto de iteración ni proyecto: el trabajo fluye de manera continua.

Principios Lean aplicados con prácticas Kanban (pregunta de parcial)

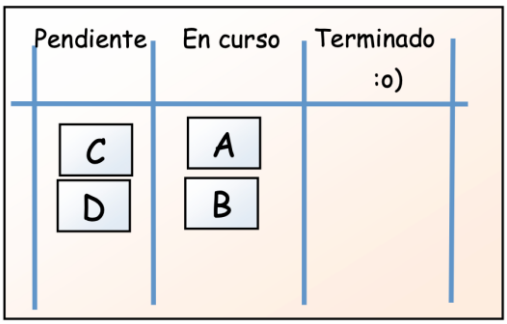
- **Eliminar desperdicios** → *visualizar el trabajo* (expone actividades sin valor y esperas que traban el flujo) y *limitar el WIP* (el trabajo a medias genera retrabajo).
- **Amplificar el aprendizaje** → *hacer explícitas las políticas* (todos acceden a las reglas de cada unidad de trabajo) y *evolución y mejora continua*.
- **Postergar decisiones hasta el momento responsable** → *limitar el WIP* (en particular el de la columna backlog).
- **Ver el todo** → *visualizar el trabajo* (el tablero muestra todo el trabajo y cómo conforma el flujo que lleva a la satisfacción del cliente).
- **Dar poder al equipo** → *políticas explícitas* y un sistema **pull**, donde cada miembro toma el trabajo cuando está en condiciones y nadie se lo impone.
- **Entregar lo antes posible** → *gestionar el flujo de trabajo* (ver los pasos de principio a fin y ajustar para entregar el mayor valor en el menor tiempo).

PREGUNTA DE PARCIAL ▸ Diferencias y similitudes entre Scrum y Kanban

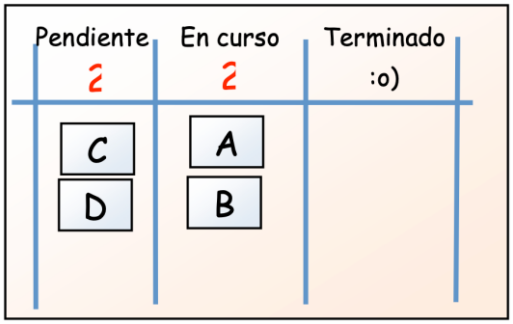
Idea central: Scrum es más **prescriptivo** (trae más reglas: roles, iteraciones fijas, compromisos). Kanban es más **flexible**: "empieza por donde estés" y agrega solo lo mínimo (visualizar, limitar el WIP, medir el flujo).

Diferencias (respuesta de examen):

- Scrum prescribe **roles** (PO, SM, desarrolladores); Kanban no prescribe ninguno.
- Scrum prescribe **iteraciones de tiempo fijo**; en Kanban se puede trabajar de forma regular o **a demanda**.
- Scrum asume **compromiso** por iteración; en Kanban es opcional.
- Métrica: Scrum usa **velocidad**; Kanban **Lead Time y Cycle Time**.
- Scrum usa gráficos **burn-down**; Kanban no necesariamente.
- En Scrum las tareas deben poder **completarse dentro de un Sprint**; en Kanban no se define tamaño.
- En Scrum el backlog **debe estar priorizado**; en Kanban no necesariamente.
- Scrum prescribe equipos **multifuncionales**; en Kanban son opcionales, y un tablero se relaciona con un **flujo de trabajo**, no necesariamente con un equipo.
- Kanban limita el **WIP directamente por estado** del flujo; Scrum lo limita **indirectamente** por iteración (no se limita en el tablero, pero existe un límite implícito porque la iteración tiene un alcance fijo).
- Scrum **se resiste a los cambios durante la iteración**; Kanban permite agregar trabajo siempre que haya capacidad.
- El tablero de Scrum **se limpia entre iteraciones**; en Kanban por lo general no (es persistente).
- Scrum prescribe **estimación y velocidad**; Kanban no.



Tablero Scrum: A y B en curso, C y D pendientes



Tablero Kanban: el WIP de cada columna es 2

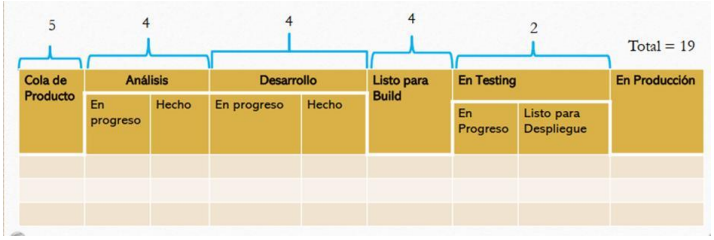
Ejemplo de la diferencia en los cambios: alguien quiere sumar un ítem **E** al tablero con A y B en curso y C y D pendientes. En **Scrum** se responde: *"no, ya estamos comprometidos con A, B, C y D; E puede entrar en la próxima iteración si el PO lo considera"*. En **Kanban** se responde: *"agregalo a la cola de producto, pero el WIP es 2 y ya hay 2 elementos, así que habría que sacar a C o D; ahora estamos con A y B y cuando terminemos trabajamos en los pendientes"*.

Similitudes (respuesta de examen): ambos son **Lean y ágiles**; emplean planificación **pull**; establecen límites de WIP (indirecto en Scrum, directo en Kanban); en ambos la **visibilidad del proceso** es la base de la mejora; buscan la **entrega temprana y frecuente**; usan equipos **autoorganizados**; necesitan **dividir el trabajo en partes**; y revisan y mejoran continuamente el plan del producto con **datos empíricos** (velocidad / tiempo de entrega).

PREGUNTA DE PARCIAL ▸ Caso: aplicación móvil para escuchar música desde el celular. Equipo: 1 analista funcional, 3 desarrolladores, 1 diseñador UX, 2 analistas de prueba y 1 responsable de despliegue. Acuerdos: el referente del cliente valida el diseño UX/UI antes de integrarlo a la funcionalidad, y se hacen revisiones técnicas al código como parte del Criterio de Hecho. (a) Diseñe el tablero completo aplicando las prácticas de Kanban (20 pts). (b) Defina los tipos de trabajo, justificando, y al menos 5 políticas de calidad (10 pts).

a) Tablero Kanban (las columnas reflejan el flujo y los acuerdos: diseño validado por el cliente y revisión de código antes de validar). Cada columna intermedia tiene dos sub-columnas (**en progreso** y **hecho**, que es la de acumulación para el sistema pull):

- **Cola de producto** (WIP = 5)
- **Análisis**: en progreso / hecho (WIP = 4)
- **Diseño**: en progreso / hecho (WIP = 2)
- **Desarrollo**: en progreso / hecho (WIP = 2)
- **Revisión**: por revisar / revisada (WIP = 3)
- **Validación**: por validar / validada
- **En producción**



Ejemplo de tablero Kanban del apunte con WIP por columna y total

b) Tipos de trabajo y justificación:

- **Desarrollo de funcionalidades** (ej.: listas de reproducción, búsquedas): es el núcleo del producto y lo que satisface al cliente y a los usuarios.
- **Corrección de errores**: problemas detectados en código o pruebas; garantiza estabilidad y funcionamiento.
- **Pruebas y validaciones**: testeo funcional y técnico; asegura la calidad antes de entregar.
- **Diseño UX/UI**: creación y refinamiento de interfaces; mejora la experiencia y satisface los criterios del cliente.
- **Despliegue y configuración**: preparar y lanzar a producción para que lleguen al usuario de forma estable.

Políticas de calidad (respuesta de examen):

1. **Reposición del tablero**: *cuándo* (cada vez que una columna tenga menos ítems que su límite mínimo), *cantidad* (la suficiente para mantener el flujo respetando el WIP) y *quién* (el equipo completo en las reuniones de planificación).
2. **Definición de actividad terminada**: para pasar al siguiente estado debe cumplirse que el **código esté revisado y aprobado por otro desarrollador**, el **UX/UI esté validado por el cliente** y las **pruebas pasen sin errores críticos**.
3. **Límites de WIP por columna** (ej.: desarrollo máx. 3, pruebas máx. 2, despliegue máx. 1): evita sobrecarga y asegura flujo continuo.
4. **Gestión de clases de servicio**: los ítems "expreso" se priorizan automáticamente; "fecha fija" tiene prioridad secundaria; "estándar" se gestiona según capacidad; "intangibles" se toman si hay capacidad ociosa. Equilibra urgencias y tareas estratégicas.
5. **Cadencia de reuniones**: planificación semanal (lunes, revisión y reposición de tareas), revisión diaria de 15 minutos para ajustar el flujo y revisión quincenal con el cliente. Mantiene la alineación y detecta bloqueos rápido.

PREGUNTA DE PARCIAL ▸ **Realice un análisis de los principios Lean y explique las prácticas que propone Kanban para aplicar en forma concreta los principios. (también hay que analizar los principios)**

Versión de examen con las 6 prácticas de Kanban (cada una apoya uno o varios de los 7 principios Lean):

1. **Visualizar el trabajo**: muestra el flujo y sus riesgos y apoya **eliminar desperdicios** al exponer cuellos de botella y trabajo innecesario; facilita decisiones colaborativas y mejora el aprendizaje continuo.
2. **Limitar el WIP**: usa la capacidad de forma eficiente y elimina desperdicios como esperas y cambios de contexto; promueve entregas rápidas y calidad.
3. **Gestionar el flujo de trabajo**: logra una entrega fluida y predecible, y refuerza **ver el todo** al mejorar la entrega de valor global.
4. **Hacer las políticas explícitas**: criterios claros y visibles que promueven coherencia, calidad y compromiso informado.
5. **Establecer ciclos de retroalimentación**: permite mejora continua y aprendizaje compartido, fomentando **amplificar el aprendizaje** y **dar poder al equipo**.
6. **Mejorar colaborativamente y evolucionar experimentalmente**: experimentos controlados para mejorar, alineados con **evitar compromisos prematuros** (diferir decisiones) y trabajar en calidad.

Cómo estudiarlo: para cada práctica preguntate *"¿qué desperdicio o problema Lean ataca?"*. Por ejemplo, limitar el WIP ataca el desperdicio de **trabajo a medias y cambios de tareas**; visualizar el trabajo ataca la **búsqueda de información y las esperas** porque todo está a la vista.

Métricas en los tres enfoques (Tradicional / Ágil / Lean)

En palabras simples: una métrica es un **número** que nos dice cómo vamos con algo que queremos controlar (el proyecto, el producto o el proceso). Si no se puede medir con un número, hay que buscar algo medible relacionado: no se mide "usabilidad" directamente, sino, por ejemplo, **cuántos clicks** se hacen en un botón (a eso se lo llama *epifenómeno*). Un ejemplo de métrica **inválida** del apunte: "cantidad de requerimientos tomados sobre los que deberíamos tomar" (no hay forma objetiva de saber cuántos "deberíamos" haber tomado).

Reglas para usar métricas: la métrica no es la escala sino la **cantidad** de cada valor (no "Bueno, Malo", sino *cuántos* son malos). Medir **tiene un costo** (planificar, calcular, analizar), así que hay que ver si el beneficio lo justifica y no exigir más precisión de la necesaria; y si se mide hay que **usar** el dato. **Nunca se usan para evaluar, premiar o castigar personas**, porque la gente empieza a "fingir" los valores y la métrica deja de reflejar la realidad. Se mide para **controlar y supervisar**, **predecir** (estimar), **evaluar** (costos) y **mejorar**.

Los tres dominios: cómo distinguirlos

Se diferencian por **sobre qué miden y quién las ve**. Para no confundirte, preguntate: *¿mido lo que se construyó, lo que cuesta/dura este proyecto, o cómo trabaja la organización en general?*

- **Producto** → mide **lo que se construye** (el software y sus artefactos). Responsables: equipo de desarrollo y Testing. Son técnicas y en tiempo real. *Ejemplo:* cantidad de defectos por severidad de la app, métodos por clase, casos de uso por complejidad.
- **Proyecto** → mide **los recursos y el plan de ese proyecto** (costos, esfuerzo, tiempo). Responsable: Líder de Proyecto. Son **privadas** del proyecto y sirven para corregir el rumbo. *Ejemplo:* costo estimado vs. costo real, fecha de entrega real vs. programada.
- **Proceso** → mide **cómo trabaja la organización** con todos sus proyectos, para mejorar a largo plazo. Responsable: Ingeniero de Procesos. Son **públicas y despersonalizadas** (no se dice "el proyecto X tuvo tantos defectos", sino "la organización tiene en promedio tanto"). Se arman **consolidando métricas de proyecto**. *Ejemplo:* promedio de defectos detectados por los usuarios en todos los productos de la empresa; desviación promedio de las estimaciones de toda la organización.

Ejemplo integrador: una empresa desarrolla apps para clínicas. En el proyecto "App de turnos" el líder mide que gastó 120 horas de las 100 estimadas (**proyecto**). El equipo de testing registra 5 defectos graves y 20 leves en esa app (**producto**). A fin de año, el Ingeniero de Procesos promedia los defectos de las 8 apps de la empresa y publica "6 defectos por app", sin nombrar proyectos (**proceso**). El mismo dato de base (defectos) cambia de dominio según **a quién le sirve y cuán agregado está**.

PREGUNTA DE PARCIAL ▸ **Dado el dominio de las métricas de software en la gestión tradicional, desarrolle una situación de ejemplo en la que usted sea el Líder de Proyecto, describa un proyecto y elabore un ejemplo propio de una métrica para cada dominio (proyecto, proceso, producto), explicando para qué la usaría.**

Proyecto de ejemplo: desarrollo de un sistema de gestión de inventarios para una cadena de tiendas minoristas.

- **Métrica de proyecto: progreso porcentual frente al cronograma.** Si el cronograma indica que al mes 3 debería estar al 50% y solo se completó el 40%, el progreso real es el 80% de lo esperado. *Uso:* detectar retrasos o adelantos y ajustar recursos o tareas para cumplir los plazos.
- **Métrica de producto: índice de estabilidad de requisitos.** Si se definieron 100 requisitos al inicio y solo 10 se modificaron durante el desarrollo, el índice es 90%. *Uso:* evaluar qué tan estable es el alcance y el impacto de los cambios.
- **Métrica de proceso: cumplimiento de hitos internos.** Si se planificaron 5 hitos en una etapa (diseño, desarrollo, pruebas iniciales…) y solo 4 se lograron en el tiempo previsto, el cumplimiento es del 80%. *Uso:* analizar la efectividad del proceso de gestión y tomar medidas contra retrasos futuros. *(Para que sea realmente de proceso conviene promediarla entre todos los proyectos de la organización y publicarla sin personalizar.)*

Enfoque tradicional

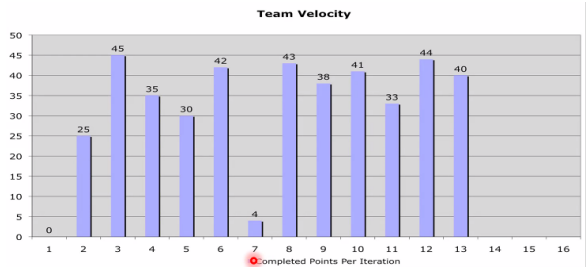
Hace énfasis en los **tres dominios** y tiene el abanico de métricas más amplio (se piensa en proyectos definidos con planificación). Es una disciplina **transversal**: al planificar se define qué métricas se usan, quién las responsabiliza y cómo se calculan. A cada rol le interesa algo distinto: a los **testers**, el esfuerzo y los defectos; al **líder de proyecto**, calendario, costos y plazos (por el contrato con el cliente).

Las **métricas básicas** (el mínimo imprescindible) son **tamaño del producto** (producto), **esfuerzo** (proyecto), **calendario** (proyecto) y **defectos** (producto), relacionadas con la triple restricción. El resumen del enfoque: **esfuerzo, tiempo, costos y riesgos**.

Ejemplo: el sistema de inventarios tiene un tamaño de 40 casos de uso (tamaño), se estimaron 800 horas y ya se usaron 500 (esfuerzo), la entrega es el 30/11 y hoy hay 2 semanas de atraso (calendario) y se encontraron 12 defectos en pruebas (defectos).

Enfoque ágil

Las métricas sirven **al equipo** y no se extrapolan a otros equipos o proyectos. Parte del principio ágil *"el software funcionando es la mejor medida de progreso"*: se mide **producto** y solo lo necesario, porque en los proyectos tradicionales se medía de todo (los tres dominios) y se perdía tiempo sin avanzar sobre el producto.



Velocidad del equipo por iteración (gráfico de barras)

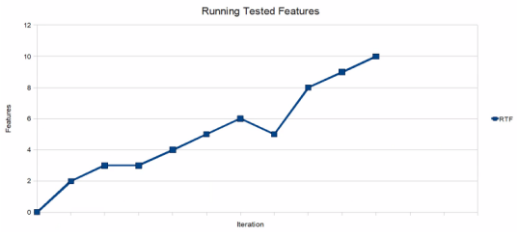
Velocidad (la más importante; métrica de producto): cuántos **puntos de historia** se completaron **y fueron aceptados por el PO** en un Sprint. Las historias parciales **no cuentan**. Se **calcula** (no se estima) después de que el PO acepta. El gráfico de barras sirve para ver si el equipo es **estable** (relacionado con el desarrollo sostenible).

Ejemplo de velocidad: en un Sprint se trabajaron cuatro US: A (5 puntos, aceptada), B (3 puntos, aceptada), C (8 puntos, **sin terminar**) y D (2 puntos, aceptada). La velocidad del Sprint es 5 + 3 + 2 = **10 puntos** (la C no suma ni "un poco"). Si en los próximos Sprints da 9, 11 y 10, el equipo es estable y se puede planificar con ~10.

Capacidad (métrica de proyecto): cuánto trabajo **puede comprometer** el equipo para el Sprint que va a empezar, en horas ideales (equipos menos maduros) o en Story Points (más maduros). Se **estima al principio** del Sprint (Sprint Planning), por eso también se llama *velocidad estimada*, y sirve para decidir **cuántas historias** tomar del Product Backlog.

Ejemplo de capacidad: 4 desarrolladores × 6 horas ideales por día × 8 días hábiles (descontando un feriado y la Planning/Review) = **192 horas ideales**. Con eso el equipo elige las US del backlog hasta llenar esa capacidad.

Cómo diferenciar velocidad y capacidad: ambas miden "cuánto trabajo", pero la **capacidad se estima antes** del Sprint (lo que *creemos* que podemos comprometer) y la **velocidad se calcula después** (lo que *realmente* se terminó y se aceptó). La velocidad pasada ayuda a estimar la capacidad futura.



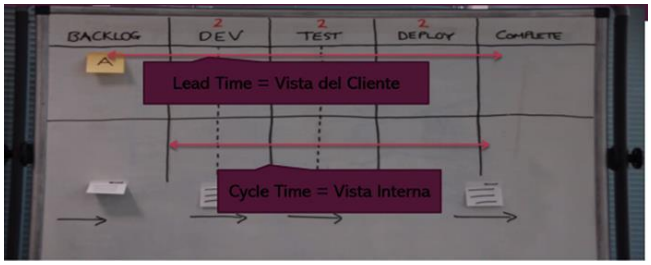
Running Tested Features: features testeadas que funcionan, por iteración

Running Tested Features (RTF): cantidad de piezas de producto (US, casos de uso, requerimientos) **terminadas, testeadas y funcionando** en un momento dado. Una curva que sube es buena; **plana o descendente indica un problema** (no se agregan características o alguna dejó de ser válida). Su debilidad: **no considera la complejidad**: una US de 1 punto cuenta igual que una de 13 (medición absoluta).

Cómo diferenciar velocidad y RTF: la **velocidad** se mide **por Sprint** y en **puntos** (pesa la complejidad); el **RTF** es un **acumulado** de **cantidad** de features funcionando. Ejemplo: Sprint 1 termina 3 features (RTF = 3); Sprint 2 termina 2 más (RTF = 5); si en el Sprint 3 se rompe una, el RTF baja a 6 en lugar de 7. Dos features pueden dar el mismo RTF pero velocidades muy distintas.

Enfoque Lean (Kanban)

El foco es el **proceso**, porque Kanban mejora un **flujo continuo** (no hay un proyecto con inicio y fin): se mide **cuánto tardan** las características en pasar por el flujo.



Lead Time (vista del cliente) vs. Cycle Time (vista interna)

- **Lead Time (Elapsed Time):** desde que el cliente pide algo (entra al Backlog) hasta que se lo entregan. Es la vista del **cliente** y la más importante para él.

- **Cycle Time:** desde que el equipo **empieza a trabajar** en la funcionalidad hasta que se entrega. Es la vista **interna**. **Cycle Time = Lead Time − tiempo en el Backlog.**
- **Touch Time:** tiempo en que las personas **efectivamente están trabajando** sobre la pieza (columnas de producción), sin las esperas (columnas de acumulación).
- **Eficiencia del ciclo de proceso = Touch Time / Lead Time.** Cuanto más cerca de **1**, menos esperas tiene el proceso.

$$Touch\ Time \leq Cycle\ Time \leq Lead\ Time$$

Relación entre los tiempos

Ejemplo concreto: el 1/10 un cliente pide "exportar reportes a PDF" (entra al Backlog). Espera 8 días hasta que el equipo la toma (9/10) y se entrega el 21/10. **Lead Time = 20 días** (1/10 a 21/10). **Cycle Time = 12 días** (9/10 a 21/10, o 20 − 8). Durante esos 12 días, en realidad se trabajó activamente solo 5 días (el resto esperó revisión o testing): **Touch Time = 5 días**. **Eficiencia = 5 / 20 = 0,25** (25%): el 75% del tiempo la tarea estaba esperando. Para mejorar se atacan las esperas, por ejemplo limitando el WIP.

Cómo diferenciar las tres rápido: *Lead* = lo que ve el **cliente** (todo el tiempo desde que pidió). *Cycle* = lo que ve el **equipo** (desde que arrancó). *Touch* = solo el tiempo **trabajado de verdad**. Siempre se cumple Touch ≤ Cycle ≤ Lead.

Métricas de producto (generales) y resumen

Hoy **no se recomienda medir en líneas de código**. Sobre defectos: **cobertura** (lo más alta posible), **defectos por severidad** (ej.: 5 defectos graves) y **densidad de defectos** (defectos por historia de usuario).



Métricas de producto: tamaño y defectos

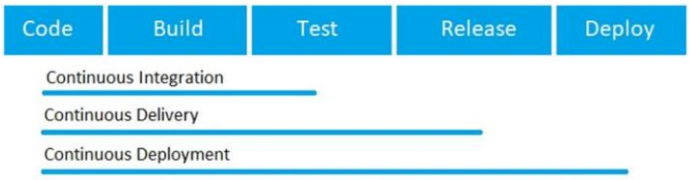
Resumen: *Tradicional* mide el **plan** (esfuerzo, tiempo, costos, riesgos) en los tres dominios. *Ágil* mide el **producto terminado por el equipo** (velocidad, capacidad, RTF). *Lean* mide el **flujo** (lead time, cycle time, touch time, eficiencia).

UNIDAD 3 – Gestión del Software como Producto: Prácticas Continuas

SCM en ambientes ágiles

En los métodos ágiles la gestión de configuración cambia de enfoque: **sirve a los miembros del equipo** (posibilita el seguimiento y la coordinación) en lugar de controlar a los desarrolladores. Como los equipos son autoorganizados, las **auditorías de configuración no son una actividad propia** de la SCM ágil y los procesos definidos del enfoque tradicional se relativizan (por ejemplo, **no existe un comité de control de cambios**). El Manifiesto Ágil se enfoca en el **proyecto**; la SCM se enfoca en el **producto** y trasciende el proyecto.

Características: responde a los cambios en lugar de evitarlos; **automatiza** donde sea posible (entrega frecuente y rápida, integración continua); da **feedback continuo** sobre calidad, estabilidad e integridad; sus actividades están **embebidas** en las tareas para lograr el objetivo del Sprint; es responsabilidad de **todo el equipo** (requiere capacitación); y adopta las **prácticas continuas**.



Integración, entrega y despliegue continuos a lo largo del pipeline

Integración Continua (CI)

Práctica en la que los desarrolladores **integran su código a un repositorio compartido varias veces al día** y cada integración se verifica con **pruebas automatizadas**, detectando problemas temprano (con integraciones frecuentes es más fácil encontrar y corregir errores). Asegura que el software **pueda desplegarse en cualquier momento**: el código compila y es de calidad. Cada desarrollador hace pruebas unitarias en su entorno (con **TDD**) y, cuando el componente funciona, lo sube al repositorio de integración; así la versión queda lista para las **pruebas de aceptación de usuario**.

Entrega Continua (Continuous Delivery)

Disciplina en la que el software se construye de modo que **puede liberarse a producción en cualquier momento**. Suma la **automatización de las pruebas de aceptación**: el producto queda listo para desplegar. **No implica liberar con cada cambio**: hay una **decisión humana** (el responsable de negocio / PO decide pasar a producción), pero la versión está lista de inmediato. **La integración continua es prerrequisito**: los artefactos del servidor de integración van a producción "con un click", por lo que se necesita **despliegue automatizado**. Despliegues más seguidos significan menos cambios por despliegue y menos riesgo. El software siempre debe estar "entregable": buildea, compila y los tests pasan.

Despliegue Continuo (Continuous Deployment)

Poner el producto en el ambiente de producción del usuario final. A diferencia de la entrega continua, **no hay intervención humana**: se usan **pipelines** con pasos que se ejecutan en orden para una instalación exitosa. Se recomienda desplegar poco después de trabajar los cambios (un error al día siguiente es más fácil de resolver porque aún recordamos lo hecho). Las **estrategias de deployment** buscan que el usuario no note que se puso una nueva versión.

- **Canary Deployment**: lanzamiento **por etapas**. Primero se implementa la actualización a una **pequeña parte de los usuarios**, que la prueban y dan feedback; una vez aceptado, se extiende al resto. Muestra cómo interactúan usuarios reales con el cambio. Ofrece actualizaciones sin tiempo de inactividad y reversiones sencillas; es más fluido que Blue-Green y las fallas tienen **impacto limitado**.
- **Blue-Green Deployment**: se **transfiere gradualmente el tráfico** de la versión anterior (entorno **azul**) a una versión nueva casi idéntica (entorno **verde**), ambas corriendo en producción. Cuando el tráfico pasó por completo a verde, el azul queda en espera por si hay que **revertir** o se retira, y se actualiza para ser la base de la próxima actualización.

Diferencia entre las tres prácticas (integración, entrega y despliegue continuos): el **nivel de automatización**.

UNIDAD 4 – Aseguramiento de Calidad de Proceso y de Producto

Conceptos generales sobre calidad

Calidad: cumplimiento de los requerimientos funcionales y de performance explícitamente definidos, de los estándares de desarrollo explícitamente documentados y de las **características implícitas** esperadas de un desarrollo profesional. Son todos los aspectos de un producto o servicio relacionados con su habilidad de satisfacer necesidades **manifiestas o implícitas** (las implícitas no se expresan). Es **subjetiva**: depende de la persona, el contexto, las circunstancias y el momento (algo puede no tener calidad para mí y sí para otro).

Gestión de calidad (tres niveles): a nivel **organización**, establece un marco de proceso y estándares que lleven a mejor software; a nivel **proceso**, aplica procesos de calidad específicos y verifica que se sigan; a nivel **proyecto**, establece un **plan de calidad**.

Aseguramiento de calidad (QA): definir procesos y estándares que lleven a productos de alta calidad. Su espíritu es **preventivo**, a diferencia del testing que "llega tarde". El equipo QA debe ser **independiente** del equipo de desarrollo para tener una perspectiva objetiva. La **planeación de calidad** produce el plan de calidad, que define qué significa "alta calidad" para el sistema y cómo se valorará. Según Humphrey incluye: introducción del proyecto, planes del producto (fechas y responsabilidades), descripciones de procesos, metas de calidad (atributos esenciales justificados) y riesgos y su gestión. Más allá de estándares y procesos, hay que desarrollar una **cultura de calidad** donde cada responsable asuma la calidad de su trabajo.

Problemas de calidad. Del *proyecto*: atrasos en las entregas, costos excedidos, trabajo fuera de hora y el fenómeno **90-90** (90% hecho, 90% restante). Del *producto*: requerimientos no claros, software que no hace lo que debería, no aplicar

SCM. Un software de calidad no solo cumple los requerimientos: también se entrega a tiempo y sin exceder costos. **El proyecto es el medio para lograr el producto: si el medio no tiene calidad, difícilmente la tenga el producto.**

Un software de calidad debe satisfacer las expectativas del cliente, del usuario, de la gerencia, del equipo de desarrollo y mantenimiento y de otros interesados.

Principios de calidad

- La calidad **no se inyecta, se embebe**: se concibe desde el momento cero (requerimientos); el testing solo la controla. **Aumentar las pruebas no aumenta la calidad.**
- Es responsabilidad de **todos** los involucrados; las **personas son la clave** (capacitación: *"si cree que la capacitación es cara, pruebe con la ignorancia"*).
- Se necesita un **sponsor gerencial** (se puede empezar por uno) y liderar con el ejemplo.
- **No se puede controlar lo que no se mide** (métricas); empezar simple, por lo básico; planificar el aseguramiento de calidad; que sea razonable para el negocio.
- **Prevenir es mejor que corregir**: corregir implica retrabajo muy costoso.

Importancia de trabajar con calidad (ventajas): es un aspecto **competitivo** (las empresas certifican estándares de *proceso*, no de producto, para competir); **retiene clientes** e incrementa beneficios (calidad = cumplir expectativas del cliente); logra equilibrio **costo-efectividad** (menos fallas, menos costos y retrabajo). Las metodologías ágiles hablan constantemente de calidad y asumen equipos capacitados; la **SCM** da el contexto base para trabajar con calidad. La **desventaja** práctica es que las organizaciones compiten para sobrevivir: pueden prometer plazos ajustados para cerrar un acuerdo y, sin tiempo suficiente, entregar menos funcionalidad o menor fiabilidad y rendimiento, afectando la calidad. La causa real de los problemas de calidad muchas veces no es mala gestión sino esa presión competitiva.

Visiones de calidad (¿calidad para quién?):

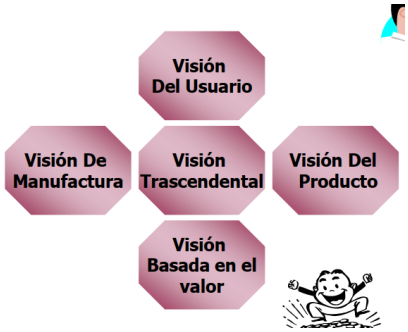
- **Del usuario**: si el producto satisface sus expectativas; es la más difícil porque está en su cabeza. El agilismo la incluye a través del **Product Owner**.
- **Del producto**: nivel de satisfacción de los requerimientos particulares.
- **Del proceso (manufactura)**: si el proceso es el correcto, aporta valor y no produce desperdicios.
- **Del valor**: equilibrio costo-beneficio, el mayor valor posible para el cliente y a la vez ganancia.
- **Trascendental**: utópica (ej.: 0 defectos), pero motiva la mejora continua.

La calidad percibida de un sistema se basa sobre todo en sus **características no funcionales**: si la funcionalidad no es la esperada el usuario "se las arregla", pero si el software es poco fiable o muy lento es casi imposible lograr sus objetivos.

Producto, proceso y proyecto: la idea completa

Analogía simple: pensá en una pizzería. El **producto** es la pizza que le llega al cliente. El **proceso** es la receta y la forma de trabajar de la cocina (qué pasos, quién hace qué, cómo se controla). El **proyecto** es lo que pasa en concreto con *un pedido*: se cocina aplicando la receta. En software: el producto es la app, el proceso es la forma de trabajar que definió la empresa (etapas, roles, cómo se testea) y el proyecto es la ejecución de ese proceso para un cliente. **El proyecto es el medio para llegar al producto**: si el proyecto sale con atrasos, sobrecostos y requerimientos poco claros, es difícil que el producto tenga calidad y, además, el cliente lo percibe mal.

¿Qué es "calidad de software" con un ejemplo? Una app de turnos médicos. Las necesidades **explícitas** son las pedidas ("el paciente puede reservar, cancelar y ver sus turnos"). Las **implícitas** son las que nadie escribió pero se esperan ("que no se cuelgue", "que la interfaz sea agradable"). Si fallan las implícitas aparece la disconformidad aunque todo lo pedido esté. Y como la calidad es subjetiva, hay que ir validando con el usuario durante todo el proyecto.



Las cinco visiones de calidad

Calidad del producto: ¿cómo se mide si cumple? Se mide contra los **requerimientos**, por eso deben ser **medibles y no ambiguos**. Ejemplo malo: *"el sistema debe ser rápido"* (no hay contra qué validar). Ejemplo bueno: *"la búsqueda de turnos debe responder en menos de 2 segundos con 100 usuarios simultáneos"*.

La gestión de calidad del producto busca equilibrar tres cosas:

- **Calidad programada**: lo que planificamos hacer.
- **Calidad realizada**: lo que realmente se construyó.
- **Calidad necesaria**: lo mínimo que el producto debe tener para satisfacer los requerimientos.

La **calidad esperada** es la zona donde las tres coinciden; lo que queda afuera es desperdicio o insatisfacción.

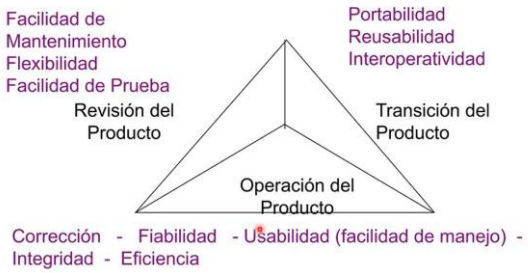
Ejemplo: se planificaron 10 funcionalidades (programada), se terminaron 8 (realizada) y el cliente necesitaba como mínimo 6 (necesaria). Las funcionalidades hechas que el cliente no necesitaba son **desperdicio**; si faltó alguna de las 6 necesarias, hay **insatisfacción**. La calidad esperada es haber hecho justamente las necesarias y las planificadas.



Calidad programada, realizada y necesaria: la intersección es la calidad esperada

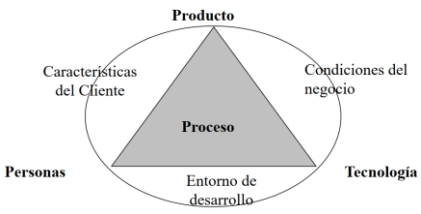
Modelos de calidad de producto: como los requerimientos cambian en cada producto, **no existe una certificación general** que diga "este software tiene tal nivel de calidad". Lo que existen son modelos que sirven para **medir características no funcionales** (RNF) y certificar la calidad en ciertos aspectos. No se profundizan en la materia, pero conviene reconocerlos:

- **ISO 25000 / 25010**: ordena la calidad del producto en características como adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad, cada una con subcaracterísticas (ver imagen).
- **McCall**: agrupa los factores en tres: **revisión del producto** (facilidad de mantenimiento, flexibilidad, facilidad de prueba), **operación del producto** (corrección, fiabilidad, usabilidad, integridad, eficiencia) y **transición del producto** (portabilidad, reusabilidad, interoperatividad), es decir, que pueda crecer en el tiempo.
- **Barbacci / SEI**: busca el **equilibrio entre performance, confiabilidad y seguridad**: mejorar un atributo puede empeorar otro (ej.: más seguridad suele reducir la performance), y se busca un punto óptimo global.



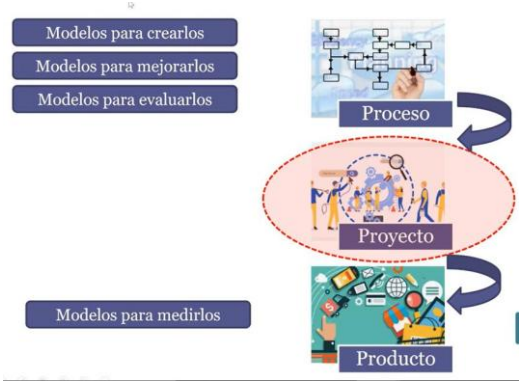
Modelo de McCall: revisión, operación y transición del producto

Calidad del proceso: ¿por qué se insiste tanto en ella? Porque lo que realmente interesa es la calidad del producto, y el **proceso es el único factor que la organización puede controlar** (*proceso con calidad → producto con calidad*). La tecnología evoluciona fuera de la empresa, las necesidades del cliente no se pueden cambiar y las personas son difíciles de controlar. Alrededor del proceso están las personas, la tecnología, las características del cliente y las condiciones del negocio, y el entorno de desarrollo.



El proceso en el centro, rodeado de factores que la organización no controla

¿Qué modelos hay para qué? Hay modelos para **crear** procesos, para **mejorarlos** (IDEAL) y para **evaluarlos** (CMMI, SPICE, ISO 9001), y otros para **medir el producto** (ISO 25010, McCall, Barbacci). Los modelos de proceso aseguran la calidad del *proceso*; los de producto miden características del *producto*.



Modelos para crear, mejorar y evaluar procesos, y para medir productos

¿Y qué controla cada actividad de calidad? Cada una tiene su momento: las **revisiones técnicas** (entre pares) revisan artefactos del producto para detectar defectos temprano; las **auditorías** (hechas por personas externas) revisan si se cumple el proceso definido y la configuración; y el **testing** controla el producto cuando ya está construido.

Ejemplo: el equipo de la app de turnos revisa el diseño en una reunión entre colegas (**revisión técnica**), un auditor de QA verifica a fin de mes si el equipo siguió el proceso definido y mantuvo actualizada la matriz de trazabilidad (**auditoría**) y los testers ejecutan casos de prueba sobre la versión terminada (**testing**).

Calidad de producto

Es el nivel de cumplimiento de las necesidades (explícitas e implícitas), que para el producto deben estar en los **requerimientos**; por eso deben ser **medibles (verificables) y objetivos (no ambiguos)**, ya que si no, no hay contra qué validar. La gestión de calidad del producto busca equilibrio entre **calidad programada** (alcance planificado), **calidad realizada** (lo realmente desarrollado) y **calidad necesaria** (mínimo para satisfacer los requerimientos); lo que queda fuera de ese equilibrio es desperdicio o insatisfacción.

Como los requerimientos varían en cada producto, **no hay modelos que acrediten el nivel de calidad de un software en general**. Existen modelos que miden RNF como epifenómenos (no se estudian en la materia): **ISO 25010**, **McCall** (agrupa factores de *revisión del producto*, *operación del producto* y *transición del producto*, es decir, que pueda crecer) y **Barbacci/SEI** (equilibrio entre performance, confiabilidad y seguridad).

Calidad de proceso

No existe un proceso que sirva para todas las organizaciones y proyectos. **La calidad de proceso nunca es un fin en sí mismo: interesa la calidad del producto** (*proceso con calidad → producto con calidad*). Se insiste en el proceso porque es **el único factor controlable**: la tecnología, las necesidades del cliente y las personas son difíciles o imposibles de controlar.

La calidad se asegura en producto y en proceso; no se habla de QA "del proyecto" porque queda implícita al implementar un proceso. Se garantiza mediante **auditorías**, y la calidad del producto se sigue con **revisiones técnicas y auditorías**, las herramientas principales. Para obtener calidad hay que definir un proceso conocido por todos que, además de tareas técnicas (requisitos, análisis, diseño, etc.), incorpore **disciplinas transversales: SCM, QA, planificación y seguimiento del proyecto**.

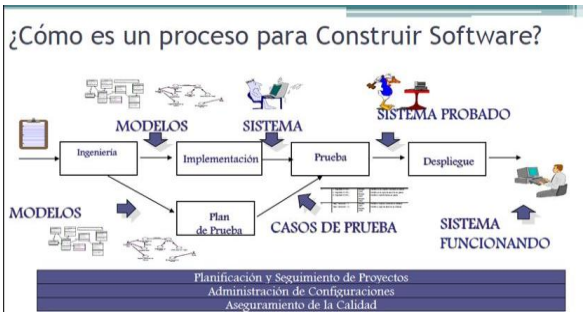
Objetivos de QA: realizar los controles apropiados del software y su proceso; asegurar el cumplimiento de estándares y procedimientos; y asegurar que los defectos del producto, proceso o estándares se informen a la gerencia.

Estándares: de **producto** (características de los componentes: estilos de programación, documentación) y de **proceso** (cómo deben implementarse requerimientos, diseño, validación, etc.). Son importantes porque (1) condensan la mejor práctica de la organización y evitan repetir errores, (2) son la base para decidir si se alcanzó el nivel de calidad requerido y (3) hacen que todos adopten las mismas prácticas, reduciendo el esfuerzo de aprendizaje.

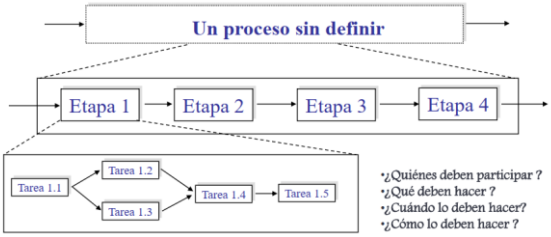
Definición de procesos: lo primero es definirlo explícitamente y comunicarlo (etapas, roles, qué, cuándo, cómo), incorporando disciplinas transversales independientemente de la metodología. Debe aportar valor y usarse **adaptado** a cada proyecto, solo en aspectos negociables (testing no es negociable).

- **Procesos definidos**: el proceso es el único factor controlable, así que la mejora continua se hace sobre él; **todos los modelos de calidad se basan en procesos definidos**.
- **Procesos empíricos**: basados en la experiencia; no creen que la calidad del proceso determine la del producto y **rechazan las auditorías** (equipos autoorganizados). Logran calidad con **revisiones técnicas** e inspección y adaptación constantes: si las personas hacen lo que deben, el producto tendrá calidad.

En palabras simples: "definir el proceso" es dejar escrito **cómo trabajamos**: etapas, quién hace cada cosa, cuándo y cómo. Sin eso, cada persona trabaja a su manera y la calidad depende de la suerte.



Un proceso para construir software: modelos, sistema, casos de prueba, y las disciplinas transversales debajo



Un proceso sin definir: ¿quiénes participan, qué deben hacer, cuándo y cómo?

Ejemplo: un equipo tiene las etapas "Análisis → Diseño → Implementación → Pruebas → Despliegue". Proceso **sin definir**: nadie sabe quién escribe los casos de prueba ni cuándo se revisa el diseño. Proceso **definido**: queda escrito que el analista funcional escribe los casos de prueba al terminar la ERS, que el diseño se revisa en una revisión técnica antes de implementar y que se hace gestión de configuración y aseguramiento de calidad **durante todo el proyecto** (las disciplinas transversales que aparecen debajo del esquema).



Ciclo de calidad de proceso: definir, desarrollar, evaluar y estandarizar o mejorar

Actividades relacionadas con el Aseguramiento de la Calidad del Software

La **administración de la calidad** busca asegurar los niveles requeridos de calidad definiendo estándares y procesos que todos respeten; su fin es generar **cultura de calidad** como responsabilidad de todos, insertando acciones que detecten lo antes posible oportunidades de mejora. El **Grupo de Aseguramiento de Calidad (GAC) no debe reportar al mismo gerente que los proyectos** (pierde independencia); debería depender del **gerente general** o de alguien realmente interesado en la calidad.

Actividades:

- **Aseguramiento de calidad**: define estándares, procesos, procedimientos y modelos sobre los que se comparará.
- **Planificación de calidad**: qué actividades y cuándo; se seleccionan (y adaptan) los estándares aplicables, los productos que deben tener calidad y sus atributos más significativos.
- **Control de calidad**: ejecución de lo planificado; asegura que el equipo respete procedimientos y estándares. Dos enfoques: **revisiones de calidad** (principal método de validación; el grupo examina parte del proceso/producto y su documentación para encontrar problemas) y **evaluaciones de software automáticas y mediciones**. Tipos de revisiones: *inspecciones para remoción de defectos* (producto), *revisiones para evaluación de progreso* (producto y proceso) y *revisiones de calidad* (producto y estándares).

Funciones de QA: prácticas de aseguramiento, evaluación de la planificación, de los requerimientos, del diseño, de las prácticas de programación, de integración y prueba, de planificación y control de proyectos, y adaptación de los procedimientos de calidad a cada proyecto. En la práctica, calidad de proceso se garantiza **definiendo estándares** (cómo hacer revisiones, administrar la configuración), **monitoreando** que se respeten y **reportando** a la administración del proyecto; sin usar prácticas inapropiadas solo porque el estándar las establece.

Dónde encaja cada técnica: los procesos se instancian en proyectos; dentro del proyecto se insertan tareas para asegurar la calidad del producto: **revisiones técnicas** (entre pares, detección temprana; se evalúa el producto, no a la persona), **auditorías de configuración funcional** (validan una versión de la línea base) y **física** (verifican una versión de la línea base) y **testing** (controla la calidad cuando el producto ya está listo). Las auditorías las hacen personas externas (los empíricos las rechazan: ven la retrospectiva como instancia de autocorrección).

Modelos de Calidad: CMMI – SPICE – ISO

Modelos para mejora de procesos: la mejora continua es entender los procesos existentes y cambiarlos para aumentar la calidad del producto o reducir costos y tiempo. Los modelos **no dicen cómo hacer las cosas: son descriptivos**. Su propósito es analizar el proceso de la organización y armar un **proyecto cuyo resultado no es un producto sino un proceso mejorado**, que se vuelca a la organización. Los **modelos de calidad para evaluar procesos** plantean una definición teórica (lineamientos) del proceso y la comparan con su ejecución en el proyecto (mediante **auditorías de proyecto**). **No son lo mismo que un estándar de proceso**: acreditan que la organización (o un área) tiene cierto nivel de madurez.

> **En palabras simples:** un modelo de calidad de procesos es como una **guía de buenas prácticas** armada con la experiencia de muchas organizaciones exitosas. **No te dice cómo hacer las cosas** (es *descriptivo*), te dice **qué objetivos tendría que cumplir** una organización madura y vos elegís cómo lograrlos. Sirve para dos cosas distintas: **mejorar** tu proceso (modelos de mejora, como IDEAL) o **evaluarlo** para ver qué tan alineado está con la guía y acreditar un nivel (modelos de calidad, como CMMI o SPICE).
>
> **Ejemplo para no confundirse:** pensá en prepararte para un examen de manejo. El **modelo de evaluación** (CMMI/SPICE) es el examen que dice "estás en nivel 2: cumplís X, Y y Z". El **modelo de mejora** (IDEAL) es el plan de estudio para pasar de nivel 2 a nivel 3: diagnosticar dónde estás, planificar, probar en un caso piloto y aprender. Un **estándar de proceso** (otra cosa distinta) sería la receta concreta de cómo debe hacerse cada paso.

¿Y ISO? (según lo que dice el apunte): ISO es una organización que publica normas. En el apunte aparecen tres: **ISO 9001**, que se usa para certificar la calidad del **proceso** (si se hace una auditoría ISO, esta te muestra las diferencias entre tu proceso definido y lo que *deberías* tener según la norma); **ISO 15504 = SPICE**, el modelo de evaluación y mejora de procesos de software; e **ISO 25000/25010**, que no es de proceso sino de **producto**: describe características no funcionales que se pueden medir. Recordá que las empresas certifican estándares de *proceso* (no de producto) para competir en el mercado internacional.

Resumen de los modelos:

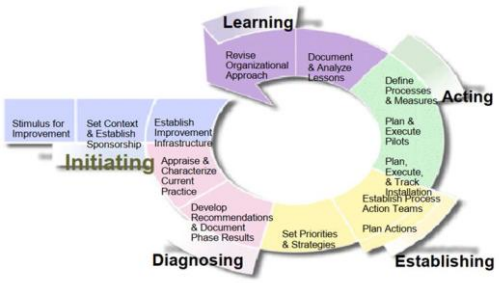
- **IDEAL**: modelo de **mejora**; es un *método* (cíclico, 5 fases) para armar el proyecto de mejora.
- **SPICE (ISO 15504)**: modelo **dual**: modelo de calidad para evaluar procesos (6 niveles de madurez) y, junto con IDEAL, para crear proyectos de mejora.
- **CMMI**: modelo de **referencia y evaluación** más usado en Argentina; evalúa madurez (por etapas) o capacidad (continuo) y se acredita con el método SCAMPI.
- **ISO 9001**: norma que certifica que el **proceso** cumple con lo exigido.
- **ISO 25010 / McCall / Barbacci**: modelos de **producto** (RNF).

Modelo IDEAL

Modelo cíclico de mejora de procesos que da el contexto para crear un proyecto cuyo resultado es un proceso definido. Se empieza buscando un **sponsor** (la mejora de procesos nunca es crítica y sin aval no suele terminar bien) y se hace un **análisis de brecha** (dónde estamos y dónde queremos ir). Sus cinco fases:

- **Initiating (Inicialización)**: se reconocen las necesidades de cambio, razones y metas; se necesita el apoyo (sponsoreo) de la organización.
- **Diagnosis (Diagnóstico)**: se establece la madurez actual y los riesgos de la mejora.
- **Establishing (Establecimiento)**: plan detallado con acciones, entregables y responsabilidades, definiendo prioridades.

- **Acting (Acción)**: se implementan los cambios en un **proyecto piloto**; si resulta satisfactorio se implanta en la empresa.
- **Learning (Aprendizaje)**: se revisa lo recolectado y se evalúan logros para que el próximo ciclo sea más efectivo; se extrapola la mejora a otros proyectos o se corrigen fallas.



Modelo IDEAL: ciclo de mejora de procesos

Modelo SPICE (ISO/IEC 15504)

Software Process Improvement Capability Evaluation. Adaptación para evaluar procesos de desarrollo por niveles de madurez, con **6 niveles**. Es un modelo **dual y teórico**: tiene un modelo de calidad (evaluación) y, junto con IDEAL, sirve para crear proyectos de mejora cuyo resultado es un proceso definido. Establece conjuntos predefinidos de procesos para definir un camino de mejora.

Nivel	Estado
Nivel 0 - Organización inmadura	La organización no tiene una implementación efectiva de los procesos
Nivel 1 - Organización básica	La organización implementa y alcanza los objetivos de los procesos
Nivel 2 - Organización gestionada	La organización gestiona los procesos y los productos de trabajo se establecen, controlan y mantienen
Nivel 3 - Organización establecida	La organización utiliza procesos adaptados basados en estándares
Nivel 4 - Organización predecible	La organización gestiona cuantitativamente los procesos
Nivel 5 - Organización optimizando	La organización mejora continuamente los procesos para cumplir los objetivos de negocio

Niveles de madurez de SPICE: de 0 (inmadura) a 5 (optimizada)

Los 6 niveles de SPICE en palabras simples: nivel 0 la organización **no tiene implementados los procesos** de forma efectiva; 1 **básica**: los procesos se implementan y logran sus objetivos; 2 **gestionada**: los procesos y sus productos de trabajo se planifican, controlan y mantienen; 3 **establecida**: se usan procesos adaptados a partir de estándares de la organización; 4 **predecible**: se miden y gestionan cuantitativamente; 5 **optimizada**: se mejoran continuamente para cumplir los objetivos de negocio.

CMMI (Capability Maturity Model Integration)

Modelo de calidad para **mejora y evaluación de procesos** de desarrollo, mantenimiento y operación de sistemas de software; conjunto de **buenas prácticas** recopiladas de organizaciones exitosas. Es de **referencia y descriptivo**: te dice qué objetivo alcanzar, vos definís qué práctica implementar. Se usa para guiar la mejora en un proyecto, división o toda la organización. La acreditación se logra evaluando de forma objetiva (comparación con el método de evaluación **SCAMPI**) y subjetiva (experiencia del evaluador). Es el más usado en Argentina.

Representación por etapas (heredera de CMM): evalúa la **madurez de toda la organización**; divide en inmaduras (nivel 1) y maduras (2 a 5). Facilita comparar organizaciones. La acreditación es **acumulativa**: se cumplen las áreas del nivel a acreditar y de los inferiores. Niveles:

Representación continua: acredita la **capacidad de cada área de proceso individualmente** (no de toda la organización), apuntando a mejorar las áreas que la organización elige. Tiene **niveles 0 a 5**; nivel 0 son los procesos que no se ejecutan (ej.: si no hace gestión de configuración, es nivel 0 en esa área).

CMMI en palabras simples: CMMI es como una **escalera de madurez** para organizaciones que hacen software. Cuanto más alto el escalón, más predecible es el resultado de sus proyectos. Ejemplo con una cocina: **nivel 1**, cada cocinero improvisa y a veces sale bien; **nivel 2**, cada pedido se planifica, se controla y se registra (se sabe qué se prometió y qué se entregó); **nivel 3**, toda la cocina usa las mismas recetas estándar y las adapta cuando hace falta; **nivel 4**, se miden tiempos y desperdicios con números y se decide con esos datos; **nivel 5**, se usa esa información para mejorar continuamente.

Los niveles con un ejemplo de software:

- **Nivel 1 – Inicial**: el equipo entrega "como puede"; si salió bien fue por el esfuerzo de una persona, y los proyectos suelen tener atrasos y sobrecostos. Ej.: un desarrollador "héroe" que se queda hasta la madrugada.

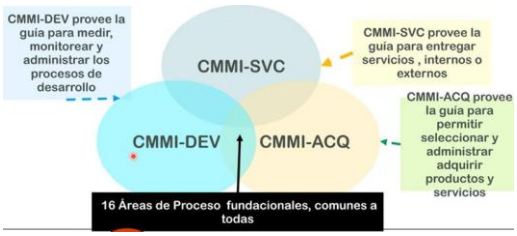
- **Nivel 2 – Administrado**: cada **proyecto** gestiona sus requisitos y planifica, mide y controla su trabajo. Ej.: hay plan, seguimiento de tareas y control de versiones en cada proyecto, aunque cada proyecto lo haga distinto.
- **Nivel 3 – Definido**: hay un **proceso estándar de la organización**, descrito en normas y procedimientos, más riguroso, con capacitación del personal y **revisiones entre pares**; los proyectos lo adaptan. Ej.: todos los proyectos usan la misma plantilla de ERS y el mismo flujo de revisión de código.
- **Nivel 4 – Cuantitativamente administrado**: se usan **métricas y estadística** para decidir y gestionar riesgos. Ej.: se sabe que, en promedio, un módulo de 10 casos de uso tiene 8 defectos, y se alerta si un proyecto se desvía mucho.
- **Nivel 5 – Optimizado**: **mejora continua** de los procesos con innovación y objetivos cuantitativos revisados. Ej.: si los datos muestran que muchos defectos nacen en requisitos, se cambia el proceso de relevamiento y se mide si mejora.

Idea clave: para acreditar un nivel hay que cumplir con ese nivel **y los anteriores** (es acumulativo). Un nivel 2 significa que la organización tiene madurez para **gestionar sus proyectos** y que el producto resultante será algo **esperable** (no una sorpresa).

CMMI-DEV y las constelaciones, con ejemplos. Una **constelación** es un conjunto de modelos, capacitaciones e información para un tipo de negocio. Son tres:

- **CMMI-DEV (Desarrollo)**: guía para **medir, monitorear y administrar los procesos de desarrollo** de una organización que *construye* productos de software. Es la constelación que se estudia, y tiene **dos representaciones** (por etapas y continua). Ej.: una fábrica de software que desarrolla una app a pedido.
- **CMMI-ACQ (Adquisición)**: guía para **seleccionar, administrar y adquirir** productos o servicios de otra empresa que tiene su propia forma de trabajo; la empresa no hace el trabajo, **contrata** a quien lo haga y controla el resultado (no es lo mismo que subcontratar personal). Ej.: un banco que contrata a una fábrica de software para desarrollar su app y debe elegirla y supervisarla.
- **CMMI-SVC (Servicios)**: guía para **entregar servicios**, internos o externos. Ej.: un equipo de soporte técnico o de operación que brinda un servicio a usuarios.

En total hay **22 áreas de proceso** (iguales en ambas representaciones), de las cuales **16 son comunes** a todas las constelaciones; el **nivel 1 no tiene áreas** (organización inmadura).



Las tres constelaciones de CMMI y sus áreas de proceso en común

Áreas de proceso del nivel 2, explicadas con un ejemplo (un **área de proceso** es un conjunto de prácticas relacionadas que, aplicadas juntas, logran objetivos importantes de mejora):

- **REQM – Gestión de requerimientos**: saber siempre qué requerimientos tiene el producto y controlar sus cambios. Ej.: la matriz de trazabilidad.
- **PP – Planificación del proyecto**: estimar y planificar. Ej.: el cronograma y el plan de release.
- **PMC – Monitoreo y control del proyecto**: comparar lo real con lo planificado y corregir. Ej.: reuniones de seguimiento y reporte de desvíos.
- **MA – Medición y análisis**: definir y usar métricas del proyecto. Ej.: esfuerzo real vs. estimado.
- **PPQA – Aseguramiento de calidad de producto y proceso**: verificar que se sigan el proceso y los estándares (disciplina transversal). Ej.: auditorías de proyecto.
- **SCM – Gestión de configuración**: control de versiones, líneas base y cambios (disciplina transversal).
- **SAM – Administración de acuerdos con el proveedor** (**opcional**): solo si se contrata a una empresa externa; hay que comprobar que cumpla el mismo nivel de calidad (proveedor **tipo A**) o tomar recaudos si no lo cumple (**tipo B**).



Áreas de proceso del nivel 2

Por etapas vs. continuo con un ejemplo. Una empresa quiere acreditar CMMI. Con la representación **por etapas** acredita un **nivel de madurez para toda la organización** (ej.: "somos nivel 2"): debe cumplir las 7 áreas del nivel 2, incluso las que no le importan tanto. Con la representación **continua** acredita la **capacidad en áreas puntuales** (niveles 0 a 5 por área): por ejemplo, decide especializarse y certificar "gestión de configuración en nivel 3", aunque en otras áreas esté en nivel 1 o 0 (si le preguntan si hace gestión de configuración y responde que no, esa área es nivel 0).

Resumen de la diferencia: *etapas* = madurez de la **organización**, 5 niveles (1 a 5), permite **comparar organizaciones** y sigue una secuencia de mejora; *continua* = capacidad de **cada área**, 6 niveles (0 a 5), permite **especializarse** y mejorar el área que se quiera.

Etapas vs. Continuo: la representación por etapas integra las formas de trabajo de toda la organización y garantiza un nivel de calidad general, pero obliga a hacer actividades en áreas que quizás no sean relevantes, y la organización se sigue considerando inmadura si no cumple el nivel 1 en todas las áreas. La continua permite **especializarse** en un área independientemente del resto y elegir qué prácticas hacer, pero pueden quedar otras áreas sin calidad. En etapas hay 5 niveles (1 a 5) que indican madurez organizacional, definidos por un conjunto de áreas y con una secuencia de mejora progresiva; en continua hay 6 niveles (0 a 5) que indican capacidad de un área, definidos por cada área.

Roles y grupos: se adaptan a las necesidades de la organización; "grupo" no es un conjunto de personas sino alguien que asuma el rol. Lo importante es que exista un responsable de cada actividad.

CMMI y Ágil

Para usar CMMI siendo ágil **ambos deben ceder**: CMMI acepta, por ejemplo, no registrar las Daily; ágil debe ceder las **auditorías** (CMMI pide monitoreo objetivo por agentes externos) y aceptar tener **trazabilidad de requerimientos** (US) que CMMI exige y ágil no. A medida que se sube de nivel no coinciden: CMMI exige definir un proceso y cumplirlo (verificado con auditorías), mientras que ágil nunca evalúa que se haya seguido el proceso definido; y desde los niveles 3 o 4 CMMI usa métricas y estadísticas de proyectos y productos para medir el proceso, mientras ágil sostiene que la experiencia no es extrapolable.

Contraste: CMMI es **descriptivo**, cuantitativo, universal, centrado en actividades, estratégico, de gestión de riesgos proactiva, orientado a mejorar la organización (uniformidad, comités, éxito por predictibilidad, reglas de atajo desalentadas). Los métodos ágiles son **prescriptivos**, cualitativos (conocimiento tácito), situacionales, centrados en el producto, tácticos ("sólo hacelo"), con gestión de riesgos reactiva, orientados al proyecto (individuos, éxito por aprovechar oportunidades, atajos alentados, conducidos por pruebas, valoran el software funcionando). **Similitudes:** buscan organizaciones de alto desempeño, ambas planean, ninguna es completa ni aplicable a cualquier proyecto, no son ideas nuevas, y ambas tienen reglas (incumplirlas tiene repercusiones).

> **En palabras simples:** CMMI y ágil buscan lo mismo (organizaciones de alto desempeño) pero con filosofías opuestas: CMMI dice **"definí un proceso, cumplilo y demostralo"** (documentación, auditorías, métricas de toda la organización); ágil dice **"confiá en el equipo, entregá software funcionando y adaptate"** (comunicación directa, equipos autoorganizados, métricas solo del equipo). Por eso, para usarlos juntos **ambos tienen que ceder un poco**.



CMMI cara a cara con ágil: verde da soporte, negro neutral, rojo desigual

La imagen clasifica las prácticas de CMMI según cómo se llevan con ágil: en **verde** las que el ágil apoya (por ejemplo definir alcance, planificar, asignar responsabilidades, capacitar a las personas, monitorear el plan y tomar acciones correctivas), en **negro** las neutrales, y en **rojo** las que **chocan**: sobre todo **monitorear objetivamente la adherencia al proceso y hacer QA** (CMMI las hace con **auditorías de agentes externos al equipo**, algo que ágil rechaza) y, en los niveles 3 y 4, **medir la performance del proceso** (ágil sostiene que la experiencia de un equipo no es extrapolable a otros).

Cómo se combinan (qué cede cada uno):

- **Ágil cede**: las **auditorías externas** y una **trazabilidad** de requerimientos (saber qué requerimientos tiene el producto en cada momento, por ejemplo vinculando las US del backlog) que el agilismo no exige.

- **CMMI cede**: acepta no tener, por ejemplo, el registro de las Daily Scrum.

¿Por qué dejan de coincidir al subir de nivel? Porque CMMI pide **definir un proceso y cumplirlo** (verificado con auditorías), mientras que en ágil **nunca se evalúa** si seguiste el proceso definido; y desde el nivel 3 o 4 CMMI usa **métricas y estadísticas de proyectos y productos** para armar una medida del proceso, mientras ágil plantea que la experiencia no es extrapolable.

Ejemplo orientativo (para entenderlo; no figura así en el apunte): un equipo Scrum que quiere acreditar nivel 2 puede mostrar su Product Backlog con trazabilidad de US (REQM), su plan de release y Sprint Planning (PP), el seguimiento con Daily y burn-down (PMC), la velocidad (MA), el repositorio con control de versiones (SCM) y las retrospectivas y la DoD (parte de PPQA). Lo que le falta y debe "ceder" es la **auditoría objetiva externa** de que realmente sigue su proceso.

CMMI	MÉTODOS ÁGILES
Medir y mejorar el proceso	Respuestas a clientes
[Mejores Procesos ↓ Mejor Producto]	Mínima sobrecarga
	Refinamiento de Requerimientos
	- Metáforas - Casos de negocio
Características de las personas	Características de las personas
- Disciplinados - Siguen reglas - Aversión al riesgo	- Comfortable - Creative - Risk Takers
Comunicación	Comunicación
- Organizacional - Macro	- Person to Person - Micro
Gestión de Conocimiento	Gestión de Conocimiento
- Activos de proceso	- Personas

Valores esenciales: CMMI frente a métodos ágiles

CMMI	MÉTODOS ÁGILES
Descriptivo	Prescriptivo
Cuantitativo	Cualitativo
- Número científicos y duros	- Conocimiento tácito
Universalidad	Situacional
Actividades	Producto
Estratégico	Táctico
"¿Cómo lo llamaremos?"	"Sólo hazlo!"
Gestión de Riesgos	Gestión de Riesgos
- Proactiva	- Reactiva

Foco de cada enfoque: descriptivo vs. prescriptivo, cuantitativo vs. cualitativo

Cómo leer esas comparaciones: CMMI es **descriptivo** (describe qué debe lograrse), **cuantitativo** (números "duros"), **universal**, centrado en **actividades**, **estratégico** ("¿cómo lo llamaremos?") y con gestión de riesgos **proactiva**; los métodos ágiles son **prescriptivos** (indican cómo trabajar), **cualitativos** (conocimiento tácito), **situacionales**, centrados en el **producto**, **tácticos** ("¡sólo hacelo!") y con gestión de riesgos **reactiva**. CMMI mejora **organizacionalmente** (uniformidad, comités, éxito por predictibilidad, desalienta atajos) y ágil mejora **en el proyecto** (individuos, tradición oral e innovación, éxito por aprovechar oportunidades, alienta atajos).

Similitudes: ambos apuntan a organizaciones de alto desempeño, ambos **planifican**, ninguno es completo ni aplicable a cualquier proyecto, no son ideas nuevas, y ambos tienen **reglas** cuyo incumplimiento tiene repercusiones.

Auditorías al software

Auditoría de calidad de software: actividad de soporte que implica una **evaluación independiente** (hecha por un grupo ajeno al equipo del proyecto) de productos o procesos para asegurar el cumplimiento de estándares, lineamientos, especificaciones y procedimientos, con criterio objetivo y documentación. Implican esfuerzo y costo, pero **sus beneficios son superiores**. Son un instrumento de QA.

Beneficios: opiniones objetivas e independientes; identificar áreas de posible insatisfacción del cliente; asegurar que se cumplen las expectativas; dar visibilidad a la gerencia sobre los procesos de trabajo; ver puntos fuertes y oportunidades de mejora; determinar que el proceso organizacional y del proyecto y las actividades de soporte se implementan de manera efectiva.

Tipos de auditorías

- **Auditoría de proyecto**: verifica que el proyecto se ejecutó con el **proceso que se definió** (adaptado al contexto). Se toman evidencias de lo que se hace y se contrastan con lo documentado (ERS, arquitectura, diseño). En Scrum el propio equipo la hace en la **Sprint Retrospective**; en tradicional el auditor debe ser externo al proyecto y a veces a

la empresa. Se rigen por el **PACS** (Plan de Aseguramiento de Calidad de Software). Su objetivo es verificar objetivamente la consistencia del producto a medida que evoluciona. Incluye:

  - **Auditoría de configuración funcional**: valida que el producto **cumpla los requerimientos**; compara el software construido (ejecutables y documentación) con la ERS, asegurando que el código implementa **solo y completamente** lo especificado. QA valida que la **matriz de rastreabilidad** esté actualizada.
  - **Auditoría de configuración física**: valida que el ítem de configuración **cumpla con la documentación técnica que lo describe**; compara el código con la documentación de soporte, para que la documentación entregada sea consistente con el código. El PACS indica quién la realiza. **El software solo se entrega cuando se arreglaron las desviaciones.**

- **Auditoría al Grupo de Calidad (externas de aseguramiento de calidad)**: hechas por personal externo a la organización para evaluar a los miembros de QA y determinar si sus evaluaciones internas se hacen correctamente.

Roles:

- **Auditado** (generalmente el líder de proyecto): propone la fecha, entrega evidencia, responde dudas, propone planes de acción para las desviaciones y responde el reporte.
- **Auditor** (una o dos personas, **externas al proyecto**; puede ser el grupo de QA): acuerda fecha, comunica el alcance, recolecta y analiza evidencia objetiva, realiza la auditoría, prepara el reporte y hace seguimiento de los planes de acción.
- **Gerente de SQA**: orientado a la gestión: prepara el plan de auditoría, calcula su costo, asigna recursos y resuelve las no-conformidades entre auditor y auditado.

Proceso de auditoría (4 etapas)

1. **Preparación y planificación**: no son sorpresa; el auditado la solicita y junto al auditor define fecha, alcance, objetivo y participantes. Se definen **métricas de auditoría**.
2. **Ejecución**: reunión de apertura (cómo será el proceso, lugar y hora) y ejecución: se responden preguntas, se muestra evidencia y se completa el **checklist**. El auditor primero escucha lo que la gente dice que hace y luego ve la documentación (lo que debería estar haciéndose).
3. **Análisis y reporte de resultados**: el auditor elabora el reporte; el auditado lo analiza (puede no estar de acuerdo con algunas prácticas, y se deja asentado); evaluación de resultados, **reunión de cierre** y entrega del reporte final.
4. **Seguimiento**: el auditado analiza las desviaciones y prepara un **plan de acción** que el auditor revisa y aprueba; el auditor puede seguir las desviaciones hasta que las considere resueltas.

Herramientas y técnicas: checklists (preguntas tipo mínimas para que el foco sea el mismo sin importar el auditor; el auditor puede preguntar más), **muestreo** (muestra representativa), **revisión de registros** y herramientas automatizadas.

Hallazgos:

- **Buenas prácticas**: algo superador a lo definido (ej.: una muy buena herramienta para gestionar el plan de proyecto).
- **Desviaciones**: lo que no se hizo o no se hizo como el proceso lo define; dos grados: **No adecuado** (realmente mal) y **Necesita mejora** (se hace pero debe mejorar). **Requieren plan de acción** del auditado.
- **Observaciones**: cosas que el auditor advierte que no llegan a desviación pero son riesgosas (ej.: mala técnica de estimación). Deberían mejorarse pero **no requieren plan de acción**.

El **reporte de auditoría** informa las desviaciones que causan disconformidad del producto respecto de sus requerimientos y cualquier desviación al proceso definido o a los requerimientos documentados. **Métricas de auditoría** (cada organización las define): esfuerzo por auditoría, cantidad de desviaciones y duración.

Auditorías vs. Revisiones técnicas: la auditoría revela **qué se hace en la realidad contra lo que se comprometió hacer** (roles: auditor, auditado, gerente de calidad; tipos: de proyecto, de calidad y de producto físicas/funcionales; hallazgos: buenas prácticas, desviaciones y observaciones). La revisión técnica busca **errores en un artefacto lo antes posible** para evitar su propagación (roles: autor, moderador, anotador, lector, inspector; formales e informales; hallazgo: errores).

PREGUNTA DE PARCIAL ▸ Propósito, características y diferencias entre testing y auditorías

En palabras simples: el **testing** pregunta *"¿el producto funciona como debe?"* y busca defectos ejecutando el software. La **auditoría** pregunta *"¿estamos trabajando como dijimos que íbamos a trabajar?"* y busca desvíos del proceso y los estándares revisando evidencia. Una mira el **producto**, la otra el **proceso** (y su aplicación en el proyecto).

Propósito y naturaleza. El testing busca **identificar defectos** para mejorar la calidad y reducir riesgos antes de liberar el producto; es una actividad **destructiva** (intenta encontrar errores). La auditoría busca **evaluar la conformidad** de productos o procesos con estándares, lineamientos y requisitos; su naturaleza es **constructiva** (asegura el cumplimiento y la mejora continua).

Objetivo y enfoque. El testing verifica y valida que el software cumpla los requisitos funcionales y no funcionales, **ejecutando casos de prueba**. La auditoría asegura la calidad del proceso y del producto final con una **opinión objetiva e independiente**, evaluando si procesos y productos cumplen los estándares predefinidos.

Momento y responsables. El testing se aplica durante todo el ciclo de vida, especialmente en implementación y pruebas, y lo hacen **testers** (y los desarrolladores en las pruebas unitarias). La auditoría suele hacerse durante o al final de fases críticas, y la hacen **auditores independientes** del equipo de desarrollo.

Resultados. Testing: defectos identificados, informes de fallas y mejora del producto antes de la entrega. Auditoría: informes de cumplimiento, **observaciones, desviaciones y buenas prácticas**, y planes de acción para resolver no conformidades.

Técnicas. Testing: caja negra y caja blanca, pruebas funcionales y no funcionales, y niveles unitario, integración, sistema y aceptación. Auditoría: auditorías de proyecto, de configuración funcional y física, revisión de registros y muestreo.

Costo y relación con la calidad. El testing representa entre el **30% y el 50%** del costo del desarrollo; no garantiza la calidad por sí solo, pero ayuda a identificar defectos que la afectan. La auditoría implica esfuerzo adicional pero detecta riesgos temprano y puede ahorrar costos; asegura la calidad al verificar el cumplimiento de estándares y procedimientos.

Conclusión: son **complementarias**: el testing se centra en encontrar defectos del **producto** y las auditorías en la calidad del **proceso** y su alineación con los estándares.

Calidad de producto: verificación y validación

Es un proceso del **ciclo de vida completo**: empieza con la revisión de requerimientos y sigue con revisiones de diseño, inspecciones de código y pruebas. **Validación**: ¿estamos construyendo el *producto correcto*? **Verificación**: ¿estamos construyendo el producto *correctamente*? Una **falla** es un error en un producto de trabajo (salida de cualquier actividad del ciclo de vida); existen por problemas de comunicación y limitaciones de memoria.

Principios: prevenir es mejor que curar; evitar es más efectivo que eliminar; la retroalimentación enseña efectivamente; priorizar lo rentable; olvidarse de la perfección (no se puede conseguir); enseñar a pescar en lugar de dar el pescado. Hay **dos aproximaciones complementarias**: revisiones técnicas y pruebas de software.

Revisiones técnicas (Peer Review)

Actividad realizada por un colega para mejorar la calidad mediante la **detección temprana de errores** en cualquier artefacto (código, requerimientos, diseño, arquitectura, riesgos, estimaciones, planes). Es un proceso **estático** de verificación y validación y **no corrige errores**. Busca evitar retrabajo y motiva a trabajar mejor. **Nunca se juzga al autor, sino al artefacto**: hay que crear una cultura de apoyo sin buscar culpables. Trascendió metodologías: el ágil la usa para **reemplazar las auditorías** y no traer a alguien externo (ej.: un Jr pide a un Sr revisar un patrón de diseño que quiere implementar).

Ventajas: pueden descubrirse muchos errores, pueden inspeccionarse versiones incompletas y se consideran otros atributos de calidad. **Desventajas:** es difícil introducir inspecciones formales; sobrecargan los costos al inicio y ahorran recién cuando el equipo gana experiencia; requieren tiempo para organizarse y "parecen" ralentizar el desarrollo.

Tipos: formales (proceso definido con roles: inspecciones de Fagan y de Gilb) e **informales** (sin proceso: walkthrough o recorrido).

Walkthrough (informal)

Técnica de análisis estático en la que un diseñador o programador guía a otros miembros y partes interesadas por el producto; los participantes preguntan y comentan sobre posibles errores o violaciones de estándares. **No hay proceso formal ni control**: reuniones informales de colegas. Objetivos: **mínima sobrecarga, capacitación de desarrolladores y rápido retorno**. No obtiene métricas ni deja registros, pero es **una de las más elegidas en ágil** (una revisión rápida entre colegas al terminar cada iteración).

Inspección (formal)

Tiene proceso formal y roles; requiere **checklist** (ayuda a recordar qué controlar), toma métricas y termina con un **reporte** para analizar los defectos. Su éxito depende de la planificación. Objetivos: descubrir errores, verificar que se alcanzan los requerimientos, garantizar el cumplimiento de estándares, lograr un desarrollo uniforme y hacer los proyectos más manejables. Son *time-boxed* y de alto esfuerzo intelectual.

Roles (algunos pueden ser asumidos por la misma persona):

- **Autor**: creador del producto; inicia el proceso eligiendo al moderador y, con él, a los demás roles.
- **Moderador**: planifica y lidera; entrega el producto a los inspectores **2 días antes** de la reunión; coordina que no haya conductas inapropiadas y hace seguimiento de los defectos.
- **Anotador**: registra los hallazgos; suele hacer el reporte.
- **Lector**: lee el producto para que nadie se disperse.
- **Inspector**: examina el producto antes de la reunión y registra su tiempo de preparación; todos pueden ser inspectores.

Etapas:

1. **Planificación**: el moderador, a pedido del autor, define lugar, duración y roles. Las reuniones **no deben superar las 2 horas**.
2. **Visión general** (opcional): el autor describe el producto.
3. **Preparación**: cada rol lee y analiza su copia para encontrar defectos potenciales (hace más productiva la reunión).
4. **Reunión de inspección**: se recolectan los defectos y se **descartan falsos positivos**; el lector lee, los inspectores comparten hallazgos y el anotador los registra. Termina con la conclusión de si se acepta o no el producto y un informe de qué se revisó, por quién, qué se descubrió y qué se concluyó.
5. **Corrección**: el autor corrige los defectos.
6. **Seguimiento**: si los defectos son graves puede haber **re-inspección**; si son simples, el autor los corrige y, a lo sumo, se reúne con el moderador para revisar las correcciones.

Definición de estándares: aspecto importante del QA (ver "Estándares" más arriba); para la gestión de calidad se usan **estándares de producto** (documentos y codificación) y **de proceso**.

Checklists para revisiones técnicas por tipo de artefacto

En palabras simples: un **checklist** es una lista de preguntas que el revisor usa para no olvidarse de controlar nada, sin importar quién haga la revisión. Se arma pensando **qué puede salir mal en ese artefacto**. Casi todos comparten un patrón: ¿cumple los **requisitos**?, ¿es **consistente** y **claro**?, ¿es **factible**?, ¿se puede **probar**?, ¿cumple los **estándares**?, ¿se pensó en **riesgos y mantenimiento**? Si te piden uno que no recordás, armalo con esas preguntas adaptadas al artefacto.

PREGUNTA DE PARCIAL ▸ Diseñe un checklist con al menos 5 preguntas para revisiones técnicas a la ARQUITECTURA.

- ¿La arquitectura satisface todos los requisitos funcionales y no funcionales documentados?
- ¿Es consistente con los estándares de la organización y las mejores prácticas de la industria?
- ¿Están todos los componentes del sistema y sus interfaces claramente definidos y documentados?
- ¿Está diseñada para facilitar las actividades de prueba?
- ¿Se identificaron y abordaron los riesgos arquitectónicos potenciales?
- ¿Fue revisada y aprobada por las partes interesadas (arquitectos, analistas, diseñadores, líderes de proyecto, probadores)?
- ¿Admite futuras necesidades de mantenimiento, escalabilidad y mejora?

PREGUNTA DE PARCIAL ▸ Diseñe un checklist con al menos 5 preguntas para revisiones técnicas al DISEÑO.

- **Alineación con requisitos**: ¿el diseño satisface todos los requisitos funcionales y no funcionales?
- **Consistencia**: ¿es consistente en todos los componentes y módulos?
- **Factibilidad**: ¿es técnicamente factible con la tecnología y los recursos actuales?
- **Modularidad**: ¿promueve la modularidad y la encapsulación?
- **Mantenibilidad**: ¿está estructurado para facilitar el mantenimiento y las actualizaciones?

- **Cumplimiento de estándares**: ¿cumple los estándares de la industria y de la organización?
- **Testabilidad**: ¿permite pruebas efectivas de todos los componentes?

PREGUNTA DE PARCIAL ▸ Diseñe un checklist con al menos 5 preguntas para revisiones técnicas a PLANES DE PROYECTO.

- **Claridad del alcance**: ¿está claramente definido y aprobado por todas las partes interesadas?
- **Asignación de recursos**: ¿los recursos están adecuadamente asignados para todas las tareas?
- **Gestión de riesgos**: ¿incluye una estrategia integral de gestión de riesgos?
- **Realismo de los tiempos**: ¿los cronogramas e hitos son realistas y alcanzables?
- **Comunicación**: ¿hay un plan claro de comunicación para las partes interesadas y el equipo?
- **Aseguramiento de calidad**: ¿incluye actividades adecuadas de aseguramiento y control de calidad?
- **Cumplimiento regulatorio**: ¿cumple las normativas y requisitos legales relevantes?

PREGUNTA DE PARCIAL ▸ Diseñe un checklist con al menos 5 preguntas para revisiones técnicas a la ERS.

- **Cobertura**: ¿están documentados todos los requisitos necesarios, sin omisiones?
- **Claridad**: ¿están claramente definidos y sin ambigüedades?
- **Consistencia**: ¿hay requisitos conflictivos entre sí?
- **Trazabilidad**: ¿cada requisito puede rastrearse hasta su origen y sus elementos de diseño?
- **Factibilidad**: ¿son técnica y económicamente factibles?
- **Es evaluable**: ¿está redactado de forma que permita pruebas efectivas?
- **Aprobación**: ¿todas las partes interesadas los revisaron y aprobaron?

PREGUNTA DE PARCIAL ▸ Diseñe un checklist con al menos 5 preguntas para revisiones técnicas al CÓDIGO FUENTE.

- **Estándares de codificación**: ¿cumple los estándares y convenciones definidos?
- **Funcionalidades**: ¿implementa correctamente la funcionalidad prevista según el diseño?
- **Legibilidad y documentación**: ¿está bien documentado y es fácil de entender?
- **Manejo de errores**: ¿maneja correctamente excepciones y condiciones de error?
- **Rendimiento**: ¿está optimizado donde es necesario?
- **Seguridad**: ¿aborda posibles vulnerabilidades?
- **Verificación**: ¿fue suficientemente probado con pruebas unitarias u otros métodos?

PREGUNTA DE PARCIAL ▸ Diseñe un checklist con al menos 5 preguntas para revisiones técnicas al PLAN DE PRUEBAS.

- **Cobertura**: ¿cubre todos los requisitos funcionales y no funcionales?
- **Claridad de los casos**: ¿los casos de prueba están definidos con resultados esperados?
- **Recursos**: ¿están asignados los recursos necesarios para las pruebas?
- **Cronograma**: ¿está alineado con el cronograma general del proyecto?
- **Seguimiento de defectos**: ¿existe un proceso para rastrear y gestionar defectos?
- **Estándares**: ¿cumple los estándares de pruebas de la industria y de la organización?
- **Pruebas basadas en riesgos**: ¿incluye estrategias para probar más a fondo las áreas de alto riesgo?

PREGUNTA DE PARCIAL ▸ Diseñe un checklist con al menos 5 preguntas para revisiones técnicas a CASOS DE USO.

- **Cobertura de requisitos**: ¿cubren exhaustivamente todos los requisitos funcionales?
- **Claridad**: ¿están escritos de forma clara e inequívoca, para un entendimiento coherente entre revisores?
- **Consistencia**: ¿son consistentes entre sí y con el resto de la documentación (ERS, diseño)?
- **Actores**: ¿están identificados correctamente y representan los roles de usuario que interactúan con el sistema?
- **Flujo de eventos**: ¿describen claramente los flujos normales y alternativos, sin omisiones?
- **Factibilidad**: ¿son técnicamente factibles dadas las limitaciones de tecnología y recursos?
- **Es evaluable**: ¿se pueden derivar casos de prueba para validar la funcionalidad?

Testing

Contexto y definición

El testing es una tarea del aseguramiento de calidad, junto con el control de calidad de proceso y producto. La calidad se obtiene a lo largo de todo el desarrollo (detectar errores temprano es menos costoso) y una buena **SCM** es el punto de partida del QA. Estándares como ISO y CMMI acreditan la calidad del *proceso*; la del *producto* no es estandarizable por la variación de requerimientos. El testing revela la calidad del software una vez desarrollado, pero **no asegura la calidad por sí solo**: **QA ≠ Testing**. El QA se aplica en todo el ciclo de vida; el testing se hace cuando el producto (o la parte a testear) ya está construido y forma parte del control de calidad.

Testing: proceso mediante el cual se somete un software o componente a condiciones específicas para determinar si es válido en función de los requerimientos. Es una actividad **destructiva**: se ejecuta un programa con la intención de **encontrar fallas**, asumiendo de antemano que hay defectos. El software debe ser predecible y consistente, sin sorpresas. Una prueba es **exitosa si encuentra defectos**; por eso un software de alta calidad lograda a lo largo del desarrollo dificulta encontrar defectos y puede llevar a un testing "no exitoso", mientras que uno de baja calidad produce mucho retrabajo e idas y vueltas con desarrollo. Es costoso: nunca se cubre el 100%, hay que buscar una **cobertura óptima** (costo-beneficio) que se empieza a definir con los **criterios de aceptación**.

Principios del testing

- Es destructivo, con actitud negativa, y el exitoso es el que encuentra defectos. Cuesta entre **30% y 50%** del valor del producto.
- **Pone en evidencia defectos pero no agrega calidad ni garantiza que no haya errores**; un desarrollo exitoso puede llevar a un testing no exitoso. Es necesario siempre y puede empezar antes de la codificación (necesita los requerimientos para armar casos de prueba).
- **Agrupamiento de defectos** (Pareto): ≈80% de los defectos están en el 20% de la funcionalidad, así que se prioriza ese conjunto.

Otros principios del testing (con su explicación):

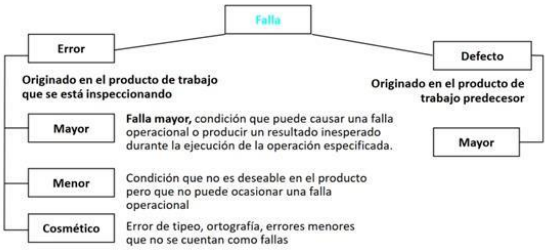
1. **Definir el resultado esperado** en cada caso de prueba; si no, un resultado erróneo se interpreta como correcto ("el ojo ve lo que quiere ver").
2. **Un programador debe evitar testear su propio programa**: no "quiere" encontrar sus defectos y puede tener malentendidos del dominio que no detectaría.
3. **Una empresa no debe testear sus propios programas**: mismo motivo, y podría destinar menos recursos para cumplir calendario y costos.
4. **Inspeccionar minuciosamente los resultados** de cada prueba: puede surgir más de un defecto por caso.
5. Escribir casos para condiciones de entrada **inválidas e inesperadas**, además de las válidas.
6. Examinar si el programa hace lo que debe es **solo la mitad**: también hay que ver que no haga lo que **no** debe.
7. **Evitar casos de prueba desechables** (ad-hoc): deben ser reproducibles y reutilizables; sin condiciones y pasos no se puede reportar un defecto.
8. No planificar el testing suponiendo que no se encontrarán errores.
9. **La probabilidad de más errores en una parte es proporcional a los ya encontrados en ella**.
10. Las pruebas son **creativas e intelectualmente desafiantes**.

Mitos: es falso que el testing demuestre que no hay errores, que un programa hace lo que debe o que muestre que funciona. Se parte de suponer que hay defectos y se busca la mayor cantidad posible. Un programa puede hacer lo que debe y aun así tener defectos (si además hace lo que no debe).

¿Cuánto testing es suficiente? El testing exhaustivo es imposible; se detiene según el **nivel de riesgo o costo** y los riesgos definen qué se prueba primero. El **criterio de aceptación** decide si una fase de testing está completa y puede definirse por costos, % de tests corridos sin fallas, inexistencia de defectos de cierta severidad, pasar el conjunto de pruebas y la cobertura estructural, **Good Enough** (cierta cantidad de fallas no críticas es aceptable) o defectos detectados similares a los estimados. Se negocia con el **PO** en ágil y con el **líder de proyecto** en tradicional.

Conceptos importantes

- **Error vs. defecto**: un **error** se detecta y corrige en la misma etapa; un **defecto** es un error que **se traslada a una etapa posterior** a la que se introdujo (el testing encuentra defectos). Ninguno implica necesariamente una **falla** (mal funcionamiento): un error de colores en la interfaz no es falla.



Error, defecto y falla: relación y severidad

- **Severidad** (técnica, la define el tester): *bloqueante* (no permite continuar), *crítico* (el sistema funciona pero la funcionalidad tiene un defecto crítico), *mayor* (funciona pero no correctamente), *menor* (funciona con advertencias erróneas o errores de poca importancia) y *cosmética* (formatos, distribución de la GUI).
- **Prioridad** (impacto en el negocio, ordena la atención): urgencia, alta, media, baja. Son **independientes**: un defecto cosmético puede ser de baja prioridad en un sistema de inventarios y de alta en uno de marketing.
- **Caso de prueba**: secuencia de pasos para obtener un resultado esperado, con **condiciones previas**. Es **el artefacto más importante** del testing: se hace una vez y sirve para infinitas pruebas con el mismo resultado esperado. Se busca minimizar su cantidad maximizando los defectos encontrados. Se compone de **objetivo, condiciones de prueba (datos de entrada y entorno) y resultado esperado**; sale de los requerimientos (permite verificar y validar).
- **Ciclo de prueba**: ejecución de un conjunto de casos sobre una versión del producto. El **ciclo 0 es siempre manual** (se configura todo); desde el ciclo 1 se pueden automatizar. Si hay muchos ciclos, probablemente el QA no fue bueno (mucho retrabajo y costo).
- **Regresión**: ejecutar **todos** los casos en cada ciclo como si fuera el ciclo 0 (ideal, pero el menos usado por costo y tiempo). La alternativa es ejecutar a partir del ciclo 1 solo los casos asociados a defectos previos, pero al corregir se suelen introducir errores en otras partes. Solución habitual: **no aplicar regresión** hasta que estén corregidos todos los defectos y entonces correr **todos los casos** una vez. La **automatización** permite hacer cuantos ciclos con regresión se quiera (el esfuerzo de automatizar equivale al ciclo 0 manual). Como el testing es caja negra, no se sabe qué afecta una corrección, así que la mejor estrategia es aplicar regresión, aunque en la práctica se usa sin regresión.

Niveles de prueba

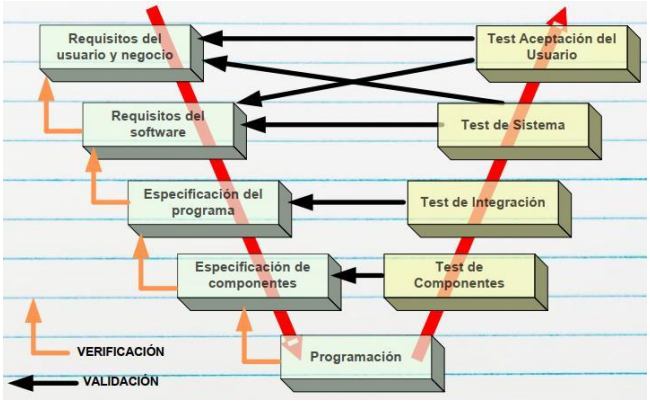
Determinan el foco o granularidad de la prueba: se empieza por componentes pequeños y se integran en pruebas de mayor granularidad (al revés de cómo se desarrollan).

- **Unitarias**: las hace el **desarrollador** y están incluidas en el **DoD**. Sobre un componente aislado, con acceso al código; los errores suelen repararse al encontrarlos sin registro formal.
- **Integración**: verifican el funcionamiento conjunto de dos o más componentes que interactúan por sus interfaces. Su resultado es el **build** (CI). Puede ser **incremental top-down** (de lo general a lo específico) o **bottom-up**.
- **Sistema o de versión**: sobre la versión de un incremento o producto, **obligatoriamente por personas ajenas al desarrollo** (razones psicológicas). Siguen un proceso sistemático, basados en casos de prueba, en un entorno lo más parecido al real; abarcan requerimientos funcionales y no funcionales; los defectos deben ser reproducibles.
- **Aceptación de usuario**: se hacen en el despliegue, debería hacerlas el **usuario** para verificar que se cumple lo que pidió; su objetivo principal **no es encontrar fallas** sino que se familiarice y confíe en el producto. Incluyen **pruebas alfa** (en laboratorio) y **beta** (en ambientes reales). En teoría el usuario arma sus pruebas; en la práctica las arma Testing. En ágil participa todo el equipo (**Sprint Review**).

> **En palabras simples:** los **niveles de prueba** responden *"¿qué tan grande es lo que estoy probando?"* (una función, varias piezas juntas, el sistema entero, la aceptación del usuario). Los **ambientes** responden *"¿dónde lo pruebo?"* (el entorno con el hardware y software necesarios). Cada nivel se hace normalmente en un ambiente distinto, y en un orden de lo **pequeño a lo grande** (al revés de cómo se desarrolla).

Ejemplo con una app para escuchar música:

- **Prueba unitaria** (nivel 1): el desarrollador prueba **una sola función**, por ejemplo `calcularDuracionPlaylist()`, con distintos valores y sin base de datos ni pantalla. Tiene acceso al código fuente, las hace él mismo y forman parte de la **DoD**. Se corrige apenas se encuentra el error, sin registrarlo formalmente. *Ambiente:* **desarrollo**.
- **Prueba de integración** (nivel 2): se verifica que **dos o más componentes** funcionen bien juntos, por ejemplo que el módulo "Playlists" le pida correctamente las canciones al módulo "Catálogo" a través de su interfaz. Su resultado es el **build** (integración continua), que es lo que se entrega a testing. Puede hacerse **top-down** (de lo general a lo específico) o **bottom-up** (al revés). *Ambiente:* generalmente **desarrollo**.
- **Prueba de sistema o de versión** (nivel 3): los **testers** (personas ajenas al desarrollo, obligatoriamente) prueban la **versión completa** como un usuario, siguiendo casos de prueba: registrarse → buscar una canción → crear una playlist → reproducir → cerrar sesión. Incluye requisitos funcionales **y no funcionales** y busca defectos reproducibles. *Ambiente:* **pruebas**.
- **Prueba de aceptación** (nivel 4): el **usuario o cliente** verifica que se cumple lo que pidió. No busca principalmente fallas sino que se **familiarice y confíe** en el producto. Hay **alfa** (el usuario prueba en un laboratorio) y **beta** (en su ambiente real). En la teoría el usuario arma sus pruebas; en la práctica las arma Testing. En ágil se hace en la **Sprint Review** con todo el equipo. *Ambiente:* **preproducción**.



Modelo en V: cada etapa de desarrollo tiene su nivel de prueba correspondiente

> **Cómo leer el modelo en V:** a la izquierda se baja desarrollando (requisitos del usuario → requisitos del software → diseño/arquitectura → programación); a la derecha se sube probando en **orden inverso**. Los requisitos del usuario se validan con la **prueba de aceptación**, los del software con la **prueba de sistema**, el diseño con la **de integración** y la programación con la **unitaria**. Sirve para saber **cuándo se puede hacer cada prueba**.

Ambientes de testing (todos los recursos de hardware y software necesarios para trabajar con el producto):

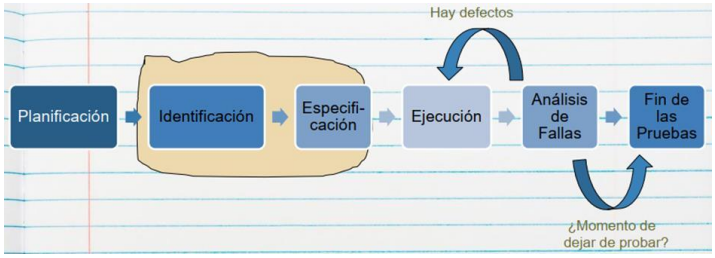
- **Desarrollo**: el entorno de los programadores (IDE, librerías, compiladores). Se hacen las pruebas **unitarias** y, generalmente, las de integración.
- **Pruebas (testing)**: usado por los **testers**; los desarrolladores **no deben tener acceso**. Se hacen las pruebas de **sistema**.
- **Preproducción**: debería ser **igual al productivo** para comprobar que el producto va a funcionar al desplegarlo. Se hacen las pruebas de **aceptación**. A veces es muy caro o imposible replicar el hardware (ej.: los servidores de Google).
- **Producción**: la configuración real de los usuarios finales. **No se prueba acá**, porque en teoría la versión ya cumple la DoD y probar en producción tiene grandes consecuencias.

Proceso de pruebas

El testing es un **proceso definido** que dura todo el ciclo de vida. El **testing ad hoc** (sin proceso, sin registrar los pasos) pierde trazabilidad. Etapas:

1. **Planificación**: cómo se incluye el testing en el plan del proyecto. Resultado: **Plan de pruebas** (riesgos y objetivos, estrategia, recursos, criterio de aceptación, entornos, quién y cuándo).
2. **Diseño**: identificar y especificar casos de prueba, datos necesarios, priorizarlos, definir entorno y si se aplica regresión; analizar si los requerimientos son testeables.
3. **Ejecución**: ejecutar los casos (automatizando lo posible), registrar y comparar resultados reales con esperados y generar el **reporte de defectos**.
4. **Análisis de fallas (evaluación y reporte)**: seguimiento de la corrección hasta cerrar todos los casos, evaluación del criterio de aceptación, reporte a interesados y lecciones aprendidas (**informe de reportes**).

5. **Fin de las pruebas**: en empresas maduras se termina cuando no hay defectos bloqueantes, críticos ni mayores y los menores/cosméticos son pocos (se anotan en la **nota de release**); informe final (opcional).



Etapas del proceso de pruebas y el ciclo de corrección de defectos

Artefactos: Plan de pruebas (análogo al plan de proyecto, pero separado; define datos, recursos, herramientas, regresión; solo necesita los requerimientos, no el código); **casos de prueba** (su característica más importante es ser **reproducibles**: si un defecto no se reproduce con un caso, no es un defecto; derivan de documentos del cliente, información relevada, requerimientos, especificaciones y código); **reporte de incidentes/defectos** (qué casos pasaron, cuáles fallaron y cómo reproducir los defectos); **informe final** (métricas, ciclos y errores por ciclo, métodos y tipos de prueba; único artefacto negociable en organizaciones informales).

Testing y ciclo de vida: en cascada las pruebas se hacen al final; en ágil (iterativo/incremental) se hace testing por iteración, con el riesgo de errores por la integración de incrementos.

Estrategias de prueba

Buscan cubrir la mayor cantidad de funcionalidades con la menor cantidad de casos. Hay dos grandes enfoques, **caja negra y caja blanca**, y lo mejor es usar una variedad de técnicas.

Cómo entender "caja blanca (estático)" y "caja negra (dinámico)": en el apunte la diferencia clave es **qué conocés del programa**: en la **caja blanca** tenés el código (o pseudocódigo/diagrama de flujo) y diseñás pruebas recorriendo sus caminos lógicos; en la **caja negra** no mirás el código, solo **entradas y salidas**. La lista de temas de la cátedra las agrupa además como estático/dinámico, que se puede entender así: lo **estático** se analiza **sin ejecutar** el sistema (revisando el código, el grafo de flujo y los caminos, como en las revisiones técnicas) y lo **dinámico** se analiza **ejecutando** el sistema con datos de entrada y comparando la salida real con la esperada. **Ejemplo:** revisar el `if` de la función de descuentos para ver todos sus caminos es caja blanca; ingresar edad 17 en el formulario y ver qué mensaje devuelve es caja negra.

Caja negra: no se conoce la estructura interna; la funcionalidad se analiza por sus **entradas y salidas**. Se clasifican en:

- **Basados en especificaciones**: *partición de equivalencias* (identificar clases de equivalencia: subconjuntos de datos que producen resultados equivalentes) y *análisis de valores límite* (variante que usa los bordes de las clases).
- **Basados en experiencia**: *adivinanza de defectos* (lista de defectos posibles o situaciones propensas a error según la intuición) y *testing exploratorio* (el tester aprende el sistema mientras prueba y genera nuevas pruebas).

Partición de equivalencias — procedimiento: (1) identificar las **condiciones externas** de entrada y salida (si hay dos datos del mismo tipo, como dos calles, son condiciones distintas); (2) identificar los **subconjuntos de valores válidos e inválidos** que dan resultados equivalentes (los inválidos tienen asociados mensajes de error; la **unión de todos los subconjuntos debe ser igual al universo** de la condición, para no olvidar ninguno); (3) armar los casos tomando **un solo valor concreto de cada subconjunto**.

Cómo escribir el caso: *prioridad* alta para caminos felices o errores críticos del negocio, baja para validaciones (valores no ingresados, mal formato); *nombre* claro que describa el escenario (ej.: "Ingresar al sitio web con edad menor a 18 años"); *precondiciones* con valores concretos (ej.: "el usuario Juan está logueado con permiso de administrador"); *pasos* sin ambigüedad, que arrancan con la opción que dispara la funcionalidad y suelen terminar en una confirmación; *resultado esperado* concreto y específico (ej.: el sistema muestra el mensaje "Debe seleccionar una fecha"), relacionado con el paso donde se produce.

Partición de equivalencias y valores frontera con un ejemplo. Un formulario de una app de música para adultos pide la **edad** y solo acepta de **18 a 65 años**.

Idea: no hace falta probar todas las edades (del 0 al 120): todas las edades dentro del mismo grupo se comportan igual. Se divide en **clases de equivalencia** y se prueba **un valor por clase**. Los errores suelen aparecer en los **bordes**, por eso el análisis de valores frontera prueba justo ahí.

- **Condición externa de entrada:** edad. **Condición de salida:** mensaje del sistema.

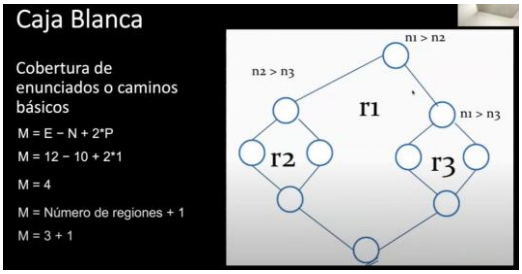
- **Clases de equivalencia:** (1) edad menor a 18 → inválida, muestra advertencia; (2) edad de 18 a 65 → válida, permite continuar; (3) edad mayor a 65 → inválida, muestra advertencia; (4) dato no numérico o vacío → inválida, muestra error. La unión de todas las clases debe cubrir **todo el universo** de la condición, para no olvidarse de ninguna.
- **Casos de prueba por partición** (un valor por clase): 10, 30, 80 y "abc".
- **Análisis de valores frontera** (los bordes de cada clase y sus vecinos): **17, 18, 65 y 66** (y a veces 19 y 64). Si el programador escribió `edad > 18` en lugar de `edad >= 18`, el caso 18 lo descubre y el caso 30 **no**.

Cómo escribirlo como caso de prueba: *Nombre:* "Ingresar al sitio con edad menor a 18 años". *Precondición concreta:* el usuario Juan está en la pantalla de registro. *Pasos:* Juan ingresa la edad 17 y presiona "Continuar". *Resultado esperado:* el sistema muestra el mensaje "Debe ser mayor de 18 años" (siempre un resultado concreto, relacionado con el paso). *Prioridad:* baja (es una validación); los caminos felices y errores críticos del negocio son de prioridad alta.

Diferencia en una frase: la **partición de equivalencias** elige *un* valor representativo de cada grupo; el **análisis de valores frontera** es una variante que elige los valores **en los límites** de esos grupos.

Caja blanca: se dispone del código (o pseudocódigo/diagrama de flujo) y se diseñan casos para maximizar defectos con la menor cantidad de casos. **Dos fallas**: el número de caminos lógicos únicos puede ser enorme, y las pruebas de camino exhaustivas no aseguran que el programa cumpla la especificación, ni detectan caminos faltantes ni errores de datos sensibles. Distintas **coberturas**:

- **Cobertura de enunciados o caminos básicos**: encontrar todos los **caminos independientes** y recorrerlos al menos una vez. Se calcula la **complejidad ciclomática (M)**, que da el **límite inferior de casos de prueba**. Procedimiento: (1) representar la funcionalidad como **grafo de flujo** (los métodos recursivos quedan fuera); (2) calcular **M = E – N + 2P** (E aristas, N nodos, P componentes conexos o nodos de salida), o **M = regiones cerradas + 1**; (3) definir el conjunto mínimo de caminos independientes (**no son casos de prueba**); (4) diseñar casos que recorran cada camino (los valores se mapean a precondiciones o entradas); (5) ejecutar y comprobar. Heurística: **M 1-10** programa simple, poco riesgo; **11-20** más complejo, riesgo moderado; **21-50** complejo, alto riesgo; **más de 50** no testeable, muy alto riesgo.



Grafo de flujo: M = regiones + 1

Complejidad Ciclomática	Evaluación del Riesgo
1-10	Programa Simple, sin mucho riesgo
11-20	Más complejo, riesgo moderado
21-50	Complejo, Programa de alto riesgo
50	Programa no testeable, Muy alto riesgo

Complejidad ciclomática y riesgo

- **Cobertura de sentencias**: mínima cantidad de casos para ejecutar **todas las sentencias** (asignaciones, invocaciones; las estructuras de control **no** son sentencias, son decisiones).
- **Cobertura de decisión**: evaluar **todas las ramas** (verdadera y falsa) de cada decisión (el IF completo); en un IF se necesitan siempre dos casos. No importa qué condiciones la forman.
- **Cobertura de condición**: evaluar **cada condición** (cada evaluación lógica dentro de la decisión, ej.: A>0) en verdadero y en falso, sin importar la rama que tome la decisión.
- **Cobertura de decisión-condición**: cubre a la vez las ramas verdadera/falsa de las decisiones y los valores verdadero/falso de las condiciones.
- **Cobertura múltiple**: evalúa el **combinatorio de todas las condiciones** en todos sus valores de verdad. Si las decisiones son dependientes entre sí hacen falta más casos (ej.: 7 en vez de 4 si fueran independientes).

1 DECISIÓN	A	F	F	V	V	V	V	V
	B	F	V	F	V	V	V	V
2 DECISIÓN	C	-	-	-	V	V	F	F
	D	-	-	-	V	F	V	F

Combinaciones de condiciones en decisiones dependientes

Tipos de prueba

- **Smoke test**: corrida rápida **antes del ciclo 0** para validar que no haya fallas groseras; evita empezar a testear formalmente si el producto no está en condiciones.
- **Funcional**: verifica que el software se comporte como lo especifica la documentación. Puede ser **basado en requerimientos** (una funcionalidad puntual, con ERS o criterios de aceptación de una US) o **en proceso de negocio** (un proceso completo, ej.: buscar, seleccionar y facturar un artículo).
- **No funcional**: foco en el "cómo" (RNF); son las más complejas por su dependencia del entorno, que debe parecerse al del cliente; lo que no se pueda probar (ancho de banda, seguridad o performance en cierta situación) se cubre con aceptación. Incluye: **performance** (tiempos de respuesta y concurrencia; debe pasarse sí o sí), **carga** (comportamiento del hardware y comunicaciones), **stress** (forzar al sistema para que falle; mide robustez y tiempo de recuperación), **mantenimiento** (documentación, facilidad de corregir defectos), **usabilidad**, **portabilidad** (distintos entornos), **fiabilidad** (poder depender del sistema), **de interfaz de usuario** y **de configuración**.

Tipos de testing funcional, con un ejemplo cada uno (el apunte los agrupa como "basado en requerimientos" y "basado en proceso de negocio"; acá se abren como en la lista de la cátedra). El **testing funcional** controla que el software se comporte como lo especifica la documentación, cumpliendo las funcionalidades definidas (se enfoca en el **"qué"**):

- **Operación normal**: se prueba el "camino feliz" con datos válidos. Ej.: reservar un turno con fecha y médico disponibles y ver que se confirma.
- **Pruebas negativas**: se prueba con datos **inválidos o inesperados** para ver que el sistema reaccione bien (el programa también debe no hacer lo que no debe). Ej.: reservar un turno con fecha pasada y ver el mensaje de error.
- **Pruebas de negocio**: se prueba un **proceso de negocio completo**, de punta a punta. Ej.: buscar un artículo, seleccionarlo, comprarlo y facturarlo.
- **Documentación**: se verifica que la documentación (manuales, ayuda, especificaciones) **coincida con lo que realmente hace el sistema**. Ej.: el manual dice que se pueden cancelar turnos hasta 24 horas antes; se prueba que sea así.

Tipos de testing no funcional con ejemplos. Se enfoca en el **"cómo"** trabaja el sistema (requerimientos no funcionales). Son las más complejas porque dependen del entorno, que debe parecerse al del cliente; lo que no se puede probar (ancho de banda, seguridad o performance en una situación real) se cubre con las **pruebas de aceptación**.

- **Rendimiento (performance)**: tiempo de respuesta y concurrencia. Ej.: la búsqueda responde en menos de 2 segundos con 500 usuarios simultáneos. Debe pasarse sí o sí.
- **Carga**: no solo la performance sino el comportamiento del **hardware** (procesadores, discos) y las comunicaciones bajo la carga esperada. Ej.: simular 10.000 usuarios y medir CPU y disco.
- **Estrés**: se **fuerza al sistema más allá de lo normal para que falle**; se ve la **robustez** y el **tiempo de recuperación**. Ej.: 100.000 usuarios a la vez y cronometrar cuánto tarda en volver a andar.
- **Usabilidad**: que sea cómodo para el usuario. Ej.: un usuario nuevo logra crear una playlist sin ayuda.
- **Mantenimiento**: si el producto puede evolucionar: hay documentación, manual de configuración y es fácil corregir un defecto.
- **Fiabilidad**: que se pueda **depender** del sistema (resultados correctos, seguridad física del software). Ej.: no pierde las playlists tras un corte de luz.
- **Portabilidad**: funciona en los distintos entornos acordados con el cliente. Ej.: Android y iOS.
- **Seguridad**: que los datos y funciones estén protegidos. Ej.: un usuario sin permiso de administrador no puede entrar al panel de administración. Es una de las características que no siempre puede probarse del todo en el ambiente de prueba.
- **Interfaces de usuario**: se prueba la GUI (suelen ser más complejas que las interfaces de comandos).

Cómo diferenciar rendimiento, carga y estrés: *rendimiento* = ¿responde en el tiempo esperado?; *carga* = ¿aguanta el volumen **esperado** y cómo se comporta el hardware?; *estrés* = ¿qué pasa cuando **supera** lo normal, hasta que falla, y cuánto tarda en recuperarse?

Testing en ambientes ágiles y TDD

En palabras simples: en ágil no se "testea al final": se **prueba en cada iteración**, con todo el equipo involucrado y la mayor parte automatizada, para saber siempre si el producto funciona y es el que el usuario necesita.

Los tres ejes (con lo que dice el apunte):

- **Automatización**: las pruebas unitarias son parte de la **DoD** y se hacen con **TDD**; la **integración continua** ejecuta pruebas automatizadas con cada integración al repositorio; la **entrega continua** suma la automatización de las **pruebas de aceptación**; y la **automatización de la regresión** permite repetir los ciclos de prueba tantas veces como se quiera (el esfuerzo de automatizar se ve como el del ciclo 0 manual, y después se ejecuta N veces). Esto responde al riesgo propio del desarrollo iterativo: que **se introduzcan errores al integrar los incrementos**.
- **Colaboración del equipo**: el equipo es **multifuncional**: desarrolladores, testers y PO trabajan juntos y la calidad es responsabilidad de todos. Se usan **revisiones técnicas** (sobre todo walkthrough, una revisión rápida entre colegas al terminar cada iteración) en lugar de auditorías externas, y el PO define los **criterios de aceptación** de las US con los que se arman las pruebas funcionales. La definición de cuándo dejar de probar se **negocia con el PO**.
- **Enfoque en la validación del producto**: se busca confirmar que se está construyendo el **producto correcto** (validación) y no solo que esté bien hecho (verificación). Por eso las pruebas de aceptación se muestran en la **Sprint Review**, con el usuario/PO y todo el equipo, y solo se considera parte del Incremento lo que cumple la **DoD**.

Ejemplo de un Sprint: el equipo toma la US "Crear playlist" con criterios de aceptación claros. Mientras desarrolla, escribe primero las pruebas unitarias (**TDD**); al subir el código, el servidor de **integración continua** corre todas las pruebas automáticas; un colega hace una **revisión técnica** rápida del código; los testers ejecutan la **regresión automatizada** y casos nuevos; y en la **Sprint Review** el PO valida con la app funcionando que cumple lo que pidió. Si algo falla, no entra al Incremento y vuelve al backlog.

TDD (Test Driven Development): primero se construye la **prueba unitaria** a partir de los requerimientos y luego se codifica el componente (si no tengo claro qué debo hacer, no puedo crear pruebas). Involucra dos prácticas:

1. **Test first development**: se escriben primero las pruebas unitarias de la característica y luego el código necesario para que pasen. Obliga a **modularizar**, con métodos o clases de **una única responsabilidad**.
2. **Refactoring**: reestructurar el código existente **sin modificar su comportamiento**, mejorando aspectos no funcionales (claridad, menor complejidad, mantenibilidad).